CreditLens: predicting credit-card payment default

Pim van den Bos

15 October 2026

Introduction and data

CreditLens asks: **using information available at the end of six months of credit-card history, how accurately can the recorded default-payment outcome in the following month be predicted?** A probability, together with its validation, helps students and analysts understand uncertainty and the consequences of choosing a classification threshold.

The UCI Default of Credit Card Clients dataset describes Taiwanese customers with April-September 2005 histories [1]. The downloaded Kaggle CSV contains 30,000 customers, 25 columns and 6,636 defaults (22.12%). All 750,000 cell values were checked against the original UCI workbook. Source URLs, file hashes and the comparison record are retained in the accompanying asset file.

Each observation is one customer. All 30,000 identifiers are unique. No values are missing, no rows are exact duplicates and no customers were excluded. The 56 matching predictor profiles across different identifiers remain separate customers. The outcome is **0 = no recorded default payment; 1 = recorded default payment** (internally No/Yes). It concerns the following month; the supplier does not fully define its operational ascertainment. It is not assumed to be an annual or regulatory default definition.

Table 1: Twenty original candidate predictors before selection.

Group	Variables	Count	Unit
Credit limit	LIMIT_BAL	1	NT dollars
Age	AGE	1	years
Repayment status	PAY_0, PAY_2-PAY_6	6	category
Bill statement	BILL_AMT1-BILL_AMT6	6	NT dollars
Previous payment	PAY_AMT1-PAY_AMT6	6	NT dollars

The six monthly columns run from September, the most recent, back to April. ID and outcome are excluded from predictors. Sex, education and marital status are omitted to focus on payment history and financial information. This choice does not establish fairness or remove every demographic proxy.

The development sample contains 23,999 customers and 5,308 defaults. Mean age is 35.5 years (SD 9.2). Median credit limit is NT$140,000 (IQR NT$50,000- 240,000). In this sample, observed default proportions are 12.7% for September code 0 and 73.4% for 3+. These are descriptive associations, not causal effects or screening criteria. The app supplies credit-limit distributions and status proportions with sample counts and exact binomial intervals.

Methods: preparation and fitted procedures

All five approaches use the same 20 candidates and customers. Preparation is fitted within the permitted training sample; no univariate significance screening is performed. Complete data require no imputation. Missing required app inputs are rejected explicitly.

Repayment codes are **nominal categories**, with -1 as reference. The expanded published descriptions are -2 = no consumption, -1 = paid in full and 0 = use of revolving credit [2, 3]. The abbreviated UCI description uses "paid duly" for -1 and omits -2 and 0. Revolving credit is not interpreted as a precisely defined minimum payment. Codes -2, -1 and 0 remain distinct; positive codes describe months of delay, with 3-9 grouped into 3+.

Within each training sample, code 1 is pooled into code 2 if fewer than 50 training observations have code 1. The final fitted mapping preserves September code 1 and pools it for August-April. This predictor-only safeguard followed a separating sparse category in an initial bootstrap pilot. All 200 final bootstrap replicates were restarted; stored checks established identical outer/inner CV encodings under the revised rule. Complete-development fits were regenerated before test evaluation. No holdout outcome chose the mapping. This historical correction and its diagnostics are retained.

Monetary predictors use the fixed transformation $g(x) = \text{sign}(x)\log(1 + |x|/1000)$, for raw NT-dollar amount x. It accommodates skewness, zeros and valid negative bills without truncation. Age stays numerical. Factors use dummy terms for the linear models.

Table 2: Primary model specifications.

Model	Specification
Full logistic	Binomial logit; all 20 candidates; additive log-odds
LDA	Empirical class priors; common covariance; scaled full-rank matrix
LASSO logistic	Binomial glmnet; alpha = 1; retain penalized coefficients
Backward AIC	Full logistic start; backward AIC (k = 2); whole factors
Probability forest	ranger; 500 trees; probability forest; ordered factor handling

Logistic regression assumes additivity on the transformed log-odds scale. Convergence, rank, finite coefficients and boundary probabilities are checked. Full-model joint likelihood-ratio tests and IQR odds contrasts are descriptive diagnostics; they do not select predictors. Odds ratios multiply odds, not probabilities. LDA's class-conditional normality and common-covariance assumptions are restrictive with these mixed categories and skewed histories [4]. Training-only rank checks identify constant/dependent design columns.

For LASSO, preparation and standardization are repeated inside each inner training fit. A decreasing, training-generated path of 100 penalties is assessed by binomial deviance (twice log loss). The **one-SE rule** chooses the largest penalty whose estimated loss is within one fold-based standard error of the minimum [5]. Here SE is the SD of five fold losses divided by $\sqrt{5}$. This favors a stronger penalty and simpler fit within that uncertainty allowance; it is separate from choosing the final model family. The minimum-loss penalty is retained as a development sensitivity analysis.

The forest tunes mtry 4, 8 or 12 and minimum node size 10 or 30 by shared inner-CV log loss; exact ties favor larger nodes, then lower mtry [6]. Its final settings are mtry=4; min.node.size=30; trees=500. Out-of-bag error is supplementary and does not choose the final family.

Methods: validation, final choice and uncertainty

Holdout validation. Seed 5 reserves approximately 20% of each outcome class: 23,999 development and 6,001 test customers, with 5,308 and 1,328 defaults. IDs are disjoint. Data-quality checks precede splitting. Detailed exploration, support ranges, presets, tuning and model selection use development data only.

Nested cross-validation. Five stratified outer development folds are shared by all families. Within each outer training sample, five shared inner folds tune LASSO and the forest. Backward selection is repeated in every outer training sample. Each development customer receives one outer-held-out probability per family. All procedures are then fitted to complete development data, using a separately saved five-fold allocation for final tuning. Split, fold, seed and diagnostic records are retained. These are forms of internal validation, not evidence of performance in a new time or population [4].

Bootstrap validation and stability. Seed 5005 generates 200 saved samples of development customers with replacement. Every replicate refits preparation, the full logistic starting model and backward AIC selection. It evaluates its fitted pipeline on both the bootstrap sample and original development sample [4]. With AUC denoted by C, mean optimism and corrected apparent AUC are

$$\widehat{O} = \frac{1}{200}\sum_{b=1}^{200}(C_{b,\text{boot}} - C_{b,\text{original}}), \qquad C_{\text{corrected}} = C_{\text{apparent}} - \widehat{O}.$$

Original-variable inclusion frequencies describe **backward-AIC stability**, not LASSO stability or causal importance. At 50% inclusion, 200 replicates give Monte Carlo SE about 3.5 percentage points. The correction is reported as a point estimate; percentiles of a fixed apparent AUC minus replicate optimism do not capture full development uncertainty.

Performance measures. Log loss is the primary probability-quality measure: it penalizes confident errors heavily. Brier score is mean squared probability error. Lower is better for both. AUC measures default/non-default ranking, counting ties as half-correct; higher is better. Probabilities are clipped to [1e-15, 1-1e-15] only for logarithms. CV summaries pool outer-held-out predictions; log loss equals the fold-size-weighted mean. A constant baseline uses enclosing-training prevalence. Calibration plots compare probabilities with observed rates in equal-count bins, with exact binomial intervals.

Why LASSO was selected. Before opening the holdout, the prespecified rule identified the forest's minimum CV log loss, 0.4324. Eligible families had loss no more than 1% above this and AUC no more than 0.01 below that family's AUC. LASSO qualified: CV loss 0.4355, AUC 0.7722. Among eligible families it required the fewest original inputs: **14**, versus 16 for backward AIC and 20 for full logistic and the forest. Remaining ties would use lower loss, then the fixed order LASSO, backward, full, LDA, forest. These are practical tolerances, not equivalence tests. Fewer required entries and an inspectable additive log-odds structure motivated the trade-off; fewer inputs do not automatically guarantee better generalization.

Locked evaluation. The selected family, fitted settings, required inputs and model hashes were locked before test evaluation. Test AUC intervals and one paired LASSO-versus-forest DeLong comparison condition on these fixed fits [4, 7]; they omit variability from repeating development. Threshold 0.50 is prespecified, with flagging when probability is at least the threshold. The app retains the development-fitted LASSO: no holdout tuning, recalibration, threshold selection or development-plus-test refitting is performed.

Results: performance and threshold interpretation

Table 3: Shared development CV and final holdout (6,001 customers).

Model	Inputs	CV loss	Test AUC (95% CI)	Test loss	Brier
Full logistic	20	0.4330	0.7642 (0.7488-0.7796)	0.4414	0.1384
LDA	20	0.4483	0.7639 (0.7485-0.7793)	0.4590	0.1426
LASSO	14	0.4355	0.7661 (0.7509-0.7813)	0.4419	0.1389
Backward AIC	16	0.4333	0.7637 (0.7482-0.7791)	0.4417	0.1385
Random forest	20	0.4324	0.7749 (0.7601-0.7898)	0.4343	0.1364
Constant baseline	0	0.5284	0.5000 (0.5000-0.5000)	0.5285	0.1723

The forest subsequently performed better on the holdout across all three reported measures: AUC 0.7749 versus 0.7661, log loss 0.4343 versus 0.4419 and Brier 0.1364 versus 0.1389 for LASSO. The paired **forest-minus-LASSO AUC advantage** is 0.0088 (95% CI 0.0003-0.0174; p = 0.0422). This supports a small ranking advantage conditional on the trained fits. The models are not established as statistically equivalent. LASSO remains implemented because of the prespecified development-performance and input-count trade-off.

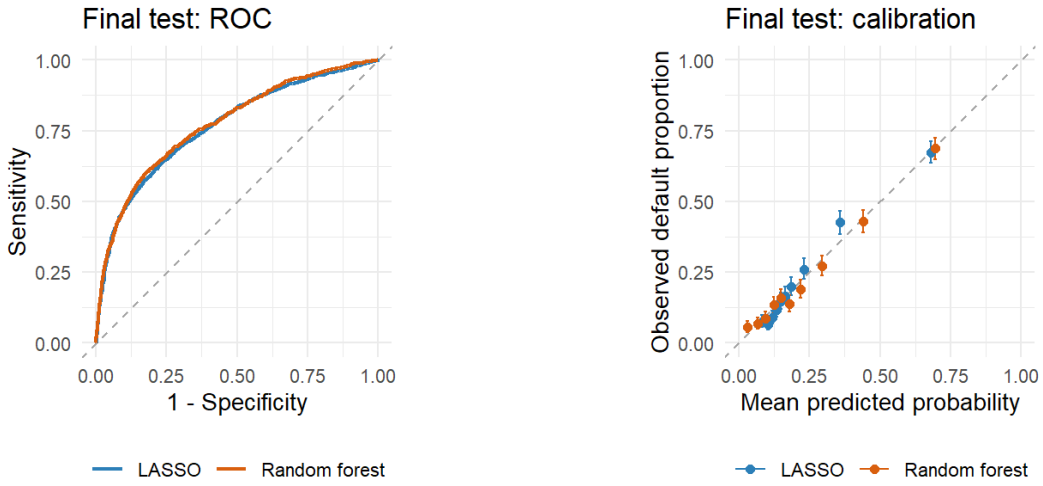


Figure 1: Fixed-model holdout ROC and calibration. Calibration points show equal-count bins with exact binomial 95% intervals; axes span 0-1.

LASSO ranks a random default/non-default pair correctly about 76.6% of the time, counting ties as half-correct. Its loss and Brier improve on the constant baseline (0.5285 and 0.1723). Calibration intercept is 0.088, slope 1.075 and offset intercept 0.000 (ideal 0, 1 and 0). Point estimates near ideal values and wide bin intervals do not establish perfect calibration; local departures remain possible.

At threshold **0.50**, the LASSO gives **416 true positives, 912 missed defaults, 215 false alarms and 4,458 true negatives**. It misses 912 of 1,328 observed defaults (**68.7%**). Sensitivity is 31.3%, specificity 95.4%, PPV 65.9% and accuracy 81.2%. Lowering the threshold flags more customers, increasing sensitivity while reducing specificity. High accuracy alone can conceal many missed defaults in this imbalanced sample. These trade-offs require decision costs and fresh validation before any operational policy.

Results: final model, importance and application

The final LASSO uses 14 original inputs: age, credit limit, six repayment statuses and six previous payments. All six bill coefficients are zero. There are 18 nonzero terms plus intercept, at lambda 0.008290814, without an unpenalized refit. The intercept is -0.7403; September category coefficients relative to -1 (paid in full) are 0.4089 for 1, 1.7918 for 2 and 1.4598 for 3+. Coefficients add on the log-odds scale. The full nonzero table, intercept, references and transformations are supplied in the asset file and app. No ordinary unpenalized p-values are attached to selected coefficients.

Table 4: LASSO-procedure importance and separate backward-AIC stability.

Input	CV permutation loss increase	Backward inclusion (%)
PAY_0	0.0741	100.0
LIMIT_BAL	0.0037	100.0
PAY_AMT1	0.0023	100.0
PAY_6	0.0023	100.0

Permutation importance averages the increase in outer-held-out log loss after shuffling an original input, using three repetitions per fold. It describes the **selected LASSO procedure**: outer-fold fits can retain different variables from the deployed complete-development fit. Correlated histories can share predictive information. Backward inclusion frequencies come from a different procedure and do not measure LASSO selection stability. Backward apparent AUC 0.7768 minus mean optimism 0.0022 yields 0.7746 (200 successful replicates; zero failures). This is slightly above backward outer-CV AUC 0.7728; the assessments use different training samples and correction procedures. Neither replaces the holdout.

Repayment effects are not constrained to increase. In the saved higher preset, holding everything else fixed, September codes 1, 2 and 3+ give 53.0%, 81.8%, 76.4%, respectively. Sparse severe-delay groups, correlated histories and unconstrained nominal effects can produce this pattern. It is not by itself a programming error, and coefficients were not manually altered. Profile changes describe predictions, not causal intervention effects.

The three-page app explains the data, makes frozen-model predictions and explores holdout performance. It highlights 14 active inputs, collapses the excluded bill candidates, and compares saved A with current B in percentage points alongside changed inputs. Lower, middle and higher presets give 9.5%, 15.5% and 53.0%; lower-to-higher is +43.5 percentage points. Required input edits clear stale results. Validation, reset, safe preset loading and extrapolation notices support interpretation. The performance page shows missed defaults, false alarms and a threshold marker on ROC; AUC, loss and Brier stay fixed as the threshold changes. PPV is undefined if nobody is flagged.

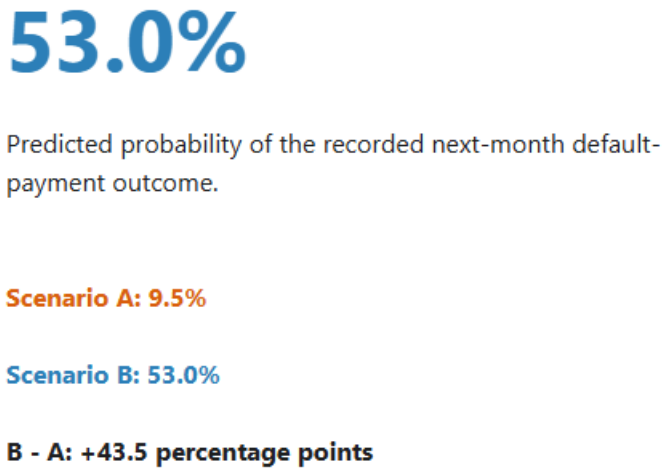


Figure 2: Predicted probability and saved-scenario comparison in CreditLens.

Public application: pimvdb.shinyapps.io/creditlens. App and report use release **CL-b2563abe35c5**, model **f9bad23a4b31**.

Discussion and conclusions

CreditLens implements a complete workflow: five model families, shared nested cross-validation, a reserved holdout and a 200-replicate backward-procedure bootstrap. The selected LASSO balances development probability performance with fewer required inputs and an inspectable additive structure. It does not have the best holdout performance: the forest performs better in ranking, log loss and Brier score. Keeping the original choice preserves the stated performance-simplicity decision instead of selecting again using test outcomes.

LASSO offers useful, imperfect discrimination and better probability quality than predicting prevalence for everyone. Calibration needs separate scrutiny; near-ideal point estimates are insufficient evidence of accurate probabilities throughout the risk range. At 0.50, high specificity coexists with missing about 69% of defaults. Users should therefore consider sensitivity, specificity, predictive value and the costs of errors together, rather than accuracy alone. Moving a threshold in the app illustrates this trade-off; it does not discover a validated lending policy.

Strengths include verified source data, no unnecessary imputation, retention of valid extreme and negative amounts, shared evaluation samples, training-only preparation and tuning, repetition of backward selection inside resampling, and a family lock preceding test evaluation. Stored fitted assets and shared prediction functions connect the report to the app. Supporting records distinguish CV importance, backward stability and final coefficients.

Limitations concern both data and modeling. The historical Taiwanese cohort does not establish performance in contemporary banking or another country. The supplier-specific outcome is incompletely defined. Grouped categories and fixed monetary transformations simplify measurement. Additive log-odds and LDA assumptions restrict the linear approaches; the forest grid is finite. Sparse severe delays and correlated monthly histories affect coefficients, selection and importance, including the non-monotonic category example. Penalized coefficients are predictive associations, not causal estimates. Omitting demographic fields does not establish fairness. Conditional holdout intervals omit variability from a new split, tuning and model-family selection.

Further work should use temporal or geographic validation, clearer outcome ascertainment, subgroup calibration checks and explicit decision costs. Recalibration, if needed for a new population, would require separate development and evaluation data. The current app is useful for teaching and communicating prediction models; its evidence does not support present-day automated credit decisions.

References

1. Yeh, I. (2009). *Default of Credit Card Clients*. UCI Machine Learning Repository. doi:10.24432/C55S3H. CC BY 4.0.
2. Chen, D. and Ye, W. (2022). *Monotonic Neural Additive Models: Pursuing Regulated Machine Learning Models for Credit Scoring*, section 4.1.1. arXiv:2209.10070.
3. H2O.ai. *Machine Learning Interpretability*, dataset coding. Tutorial.
4. Groothuis-Oudshoorn, K. and Mikhal, J. (2026). *Applied Statistical Learning*: supplied lectures on model development, validation, categorical regression and calibration.
5. Hastie, T. et al. *An introduction to glmnet*. Documentation.
6. Wright, M. N. et al. *ranger*. Reference.
7. DeLong, E. R., DeLong, D. M. and Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated ROC curves. *Biometrics*, 44, 837-845. doi:10.2307/2531595.

R code appendix

```
# CreditLens - Pim van den Bos ------------------------------------------
# Keep this file, creditlens_assets.rds, individual_assignment.Rmd and the PDF
# together. All paths below are relative to that four-file directory.
# Source loads functions only: it never trains, installs packages or opens Shiny.
#
# 1. Frozen app: source('CreditLens.R'); creditlens('app')
#    Terminal: Rscript CreditLens.R app
# 2. Report: source('CreditLens.R'); creditlens('report')
#    Or rmarkdown::render('individual_assignment.Rmd') in RStudio.
# 3. Rebuild (expensive): rebuild_analysis('path/to/UCI_Credit_Card.csv',
#      work_dir = 'analysis_work')
#    The original 30,000-row UCI/Kaggle dataset is a prerequisite for rebuilding,
#    not a submission deliverable. Rebuilding creates internal results/caches.
#    Then package_release('analysis_work', output_dir = '.') and render again.
#    Existing caches are checked; use a new work_dir for changed analysis code.
# 4. Verify: creditlens('verify'); inspect_creditlens('coefficients')
#    inspect_creditlens() lists the supporting records inside the asset file.
# 5. Deploy frozen app: creditlens('deploy')
#    Uses rsconnect's configured shinyapps.io account (outside these files).
#    No tokens are printed or included. Generated deployment files are temporary.
#
# Software: R >= 4.5; tested with R 4.5.3. Runtime: shiny, bslib, htmltools,
# ggplot2, glmnet, Matrix, pROC, digest, base64enc. Rebuild also uses MASS,
# ranger, jsonlite and readxl (original-workbook provenance check). Rendering
# needs rmarkdown, knitr, Pandoc >= 2 and a LaTeX distribution (e.g. TinyTeX).
# Deployment additionally needs rsconnect and the configured hosting account.
# Install missing packages explicitly with install.packages() before running.
# Tested versions are available with inspect_creditlens('dependencies').
# Pandoc is normally supplied/discovered by RStudio or PATH; alternatively set
# RSTUDIO_PANDOC to your own Pandoc directory. No installation path is required.
# No renv.lock, raw data, development folders or browser runner is needed for
# frozen app startup, rendering, verification or deployment.
#
# package_release() is the single release operation. It verifies current saved
# analysis outputs, records original training provenance separately from current
# source checksums, and writes the RDS used by BOTH app and report. UI/text edits
# require repackaging, not refitting. Never edit old model/manifest fingerprints.

.creditlens_file <- local({
  paths <- lapply(sys.frames(), function(frame) frame$ofile)
  paths <- Filter(
    function(path) {
      is.character(path) &&
        length(path) ==
          1L
    }, paths
  )
  if (length(paths)) {
    paths[[length(paths)]]
  } else {
    argument <- grep("^--file=", commandArgs(), value = TRUE)
    if (length(argument)) {
      sub("^--file=", "", argument[1])
    } else {
      "CreditLens.R"
    }
  }
})
.creditlens_file <- normalizePath(.creditlens_file, winslash = "/", mustWork = TRUE)
.creditlens_root <- dirname(.creditlens_file)

# 1. Fixed specification and file utilities -----------------------------

candidate_variables <- c(
  "LIMIT_BAL", "AGE", "PAY_0", paste0("PAY_", 2:6),
  paste0("BILL_AMT", 1:6),
  paste0("PAY_AMT", 1:6)
)
status_variables <- c("PAY_0", paste0("PAY_", 2:6))
money_variables <- c(
  "LIMIT_BAL", paste0("BILL_AMT", 1:6),
  paste0("PAY_AMT", 1:6)
)
status_levels <- c("-1", "-2", "0", "1", "2", "3+")
families <- c("full", "lda", "lasso", "backward", "forest")
model_labels <- c(
  full = "Full logistic", lda = "LDA", lasso = "LASSO", backward = "Backward AIC",
  forest = "Random forest", baseline = "Constant baseline"
)
config <- list(
```

```
  schema = 1L, candidates = candidate_variables, status_levels = status_levels,
  rare_status_min = 50L, split_seed = 5L, outer_seed = 1000L, inner_seeds = 1001:1005,
  final_inner_seed = 2001L, bootstrap_seed = 5005L, bootstrap_B = 200L, trees = 500L,
  threads = 4L, forest_grid = expand.grid(
    mtry = c(4L, 8L, 12L),
    min.node.size = c(10L, 30L)
```

```
  ),
  logloss_tolerance = 1.01, auc_tolerance = 0.01
)
save_atomic <- function(object, path) {
  dir.create(
    dirname(path),
    recursive = TRUE, showWarnings = FALSE
  )
  tmp <- paste0(path, ".tmp")
  saveRDS(object, tmp)
  if (file.exists(path)) {
    unlink(path)
  }
  stopifnot(file.rename(tmp, path))
}
write_json <- function(object, path) {
  jsonlite::write_json(object, path, pretty = TRUE, auto_unbox = TRUE, digits = 15, null = "null")
}


# 2. Import, audit and development descriptions --------------------

read_credit <- function() {
  files <- list.files("data", pattern = "[.]csv$", full.names = TRUE, ignore.case = TRUE)
  if (length(files) !=
    1L) {
    stop("Expected one unambiguous raw credit CSV.")
  }
  raw <- utils::read.csv(files, check.names = FALSE)
  original <- names(raw)
  names(raw) <- toupper(gsub("[ .]+", "_", names(raw)))
  outcome <- grep(
    "^DEFAULT.*NEXT.*MONTH$", names(raw),
    value = TRUE
  )
  stopifnot(
    length(outcome) ==
      1L
  )
  names(raw)[names(raw) ==
    outcome] <- "DEFAULT"
  stopifnot(
    nrow(raw) >
      0L, ncol(raw) ==
      25L, setequal(
      names(raw),
      c("ID", candidate_variables, "SEX", "EDUCATION", "MARRIAGE", "DEFAULT")
    ),
    !anyNA(raw),
    all(raw$DEFAULT %in% 0:1),
    !anyDuplicated(raw$ID),
    !anyDuplicated(raw),
    length(candidate_variables) ==
      20L
  )
  raw$DEFAULT <- factor(
    raw$DEFAULT, c(0, 1),
    c("No", "Yes")
  )
  validate_raw(raw)
  list(
    raw = raw, file = files, hash = digest::digest(files, file = TRUE, algo = "sha256"),
    name_map = data.frame(source = original, normalized = names(raw))
  )
}
make_dictionary <- function() {
  months <- c("September", "August", "July", "June", "May", "April")
  data.frame(
    variable = c("ID", candidate_variables, "SEX", "EDUCATION", "MARRIAGE", "DEFAULT"),
    role = c(
      "Identifier", rep("Candidate", 20),
      rep("Excluded predictor", 3),
      "Outcome"
    ),
```

```
    type = c(
      "Integer", "Numerical", "Numerical", rep("Factor", 6),
      rep("Numerical", 12),
      rep("Categorical", 3),
      "Binary factor No/Yes"
    ),
    units = c(
      "none", "NT dollars", "years", rep("source category", 6),
      rep("NT dollars", 12),
      rep("source category", 3),
      "next-month 0/1"
    ),
    month = c(
      "", "", "", rep(
        paste(months, "2005"),
        3
      ),
```

```
      rep("", 4)
    ),
    description = c(
      "Integrity and joins only; excluded from model and app", "Credit limit including supplementary credit",
      "Age", paste(
        months, "repayment: -2 no consumption; -1 paid in full; 0 revolving; positive delay months"
      ),
      paste(months, "bill statement"),
      paste(months, "previous payment"),
      "Sex: omitted from primary analysis", "Education: omitted from primary analysis",
      "Marital status: omitted from primary analysis", "Recorded default payment in following month; supplier-specific definition"
    )
  )
}
audit_credit <- function(input) {
  d <- input$raw
  numerical <- setdiff(
    names(d),
    "DEFAULT"
  )
  ranges <- do.call(
    rbind, lapply(
      numerical, function(v) {
        data.frame(
          variable = v, missing = sum(is.na(d[[v]])),
          minimum = min(d[[v]]),
          q01 = quantile(d[[v]], 0.01),
          q25 = quantile(d[[v]], 0.25),
          median = median(d[[v]]),
          q75 = quantile(d[[v]], 0.75),
          q99 = quantile(d[[v]], 0.99),
          maximum = max(d[[v]]),
          zero = sum(d[[v]] == 0),
          negative = sum(d[[v]] < 0),
          row.names = NULL
        )
      }
    )
  )
  cats <- c("SEX", "EDUCATION", "MARRIAGE", status_variables, "DEFAULT")
  frequencies <- do.call(
    rbind, lapply(
      cats, function(v) {
        t <- table(d[[v]])
        data.frame(
          variable = v, code = names(t),
          n = as.integer(t)
        )
      }
    )
  )
  a <- list(
    file = input$file, bytes = file.info(input$file)$size,
    sha256 = input$hash, rows = nrow(d),
    columns = ncol(d),
    observed_columns = names(d),
    events = sum(d$DEFAULT == "Yes"),
    non_events = sum(d$DEFAULT == "No"),
    missing = sum(is.na(d)),
    unique_ids = length(unique(d$ID)),
    repeated_ids = sum(duplicated(d$ID)),
    exact_duplicates = sum(duplicated(d)),
    duplicate_profiles = sum(
      duplicated(
        d[setdiff(
          names(d),
          c("ID", "DEFAULT")
        )]
      )
    ),
    duplicate_profiles_and_outcome = sum(
      duplicated(
        d[setdiff(
          names(d),
          "ID"
        )]
      )
    ),
    exclusions = 0L, corrections = 0L, duplicate_decision = "Keep distinct customer IDs even when profiles match",
    status_ambiguity = "Expanded published labels: -2 no consumption; -1 paid in full (UCI paid duly); 0 revolving credit;
↪  categories kept distinct",
    undocumented_education = sum(!d$EDUCATION %in% 1:4),
    undocumented_marriage = sum(!d$MARRIAGE %in% 1:3),
    source = "https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients",
    downloaded_from = "https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset",
    source_checked = "2026-10-14", download_date = "Not supplied", license = "CC BY 4.0"
  )
  write_json(a, "results/audit.json")
  utils::write.csv(ranges, "results/audit_ranges.csv", row.names = FALSE)
  utils::write.csv(frequencies, "results/audit_categories.csv", row.names = FALSE)
  utils::write.csv(input$name_map, "results/column_mapping.csv", row.names = FALSE)
```

```
  utils::write.csv(make_dictionary(), "results/dictionary.csv", row.names = FALSE)
  a
}
describe_development <- function(d) {
  nums <- setdiff(candidate_variables, status_variables)
  s <- do.call(
    rbind, lapply(
      nums, function(v) {
        data.frame(
          variable = v, n = nrow(d),
          missing = sum(is.na(d[[v]])),
          mean = mean(d[[v]]),
          SD = sd(d[[v]]),
          median = median(d[[v]]),
          Q1 = quantile(d[[v]], 0.25),
          Q3 = quantile(d[[v]], 0.75),
          row.names = NULL
        )
      }
    )
  )
  utils::write.csv(s, "results/development_numeric.csv", row.names = FALSE)
  cats <- do.call(
    rbind, lapply(
      c(status_variables, "DEFAULT"),
      function(v) {
        z <- if (v %in% status_variables) {
          group_status(d[[v]])
        } else {
          d[[v]]
        }
        t <- table(z)
        data.frame(
          variable = v, level = names(t),
          n = as.integer(t),
          percent = 100 * as.integer(t) / nrow(d)
        )
      }
    )
  )
  utils::write.csv(cats, "results/development_categories.csv", row.names = FALSE)
  dir.create("results/figures", showWarnings = FALSE)
  p <- ggplot2::ggplot(d, ggplot2::aes(DEFAULT, LIMIT_BAL / 1000, fill = DEFAULT)) +
    ggplot2::geom_boxplot(outlier.alpha = 0.1) +
    ggplot2::theme_minimal() +
    ggplot2::scale_fill_manual(values = c("#2C7FB8", "#D95F0E")) +
    ggplot2::labs(
      x = "Recorded next-month default", y = "Credit limit (thousand NT dollars)",
      title = "Development sample", fill = NULL
    )
  ggplot2::ggsave("results/figures/development_limit.png", p, width = 6, height = 3.5, dpi = 180)
  z <- factor(
    group_status(d$PAY_0),
```

```
    levels = status_levels
  )
  rates <- do.call(
    rbind, lapply(
      levels(z),
      function(g) {
        y <- as.integer(d$DEFAULT[z == g] == "Yes")
        ci <- stats::binom.test(
          sum(y),
          length(y)
        )$conf.int
        data.frame(
          status = g, n = length(y),
          events = sum(y),
          rate = mean(y),
          lower = ci[1], upper = ci[2]
        )
      }
    )
  )
  rates$status <- factor(rates$status, levels = status_levels)
  utils::write.csv(rates, "results/development_status_rates.csv", row.names = FALSE)
  p <- ggplot2::ggplot(rates, ggplot2::aes(status, rate)) +
    ggplot2::geom_col(fill = "#2C7FB8") +
    ggplot2::geom_errorbar(
      ggplot2::aes(ymin = lower, ymax = upper),
      width = 0.15
    ) +
    ggplot2::geom_text(
      ggplot2::aes(label = paste0("n=", n)),
      vjust = -0.8, size = 3
    ) +
    ggplot2::theme_minimal() +
    ggplot2::labs(
      x = "September repayment category", y = "Observed default proportion", title = "Development sample; exact binomial 95%
      ↪  intervals"
    )
  ggplot2::ggsave(
```

```
    "results/figures/development_status.png", p,
    width = 6, height = 3.5, dpi = 180
  )
}


# 3. Training-only preparation and resampling --------------------

signed_log_amount <- function(x) {
  sign(x) *
    log1p(abs(x) / 1000)
}
group_status <- function(x) {
  z <- as.character(x)
  if (anyNA(x) ||
    any(
      !z %in% c(
        as.character(-2:9),
        "3+"
      )
    )) {
    stop("Unsupported repayment category. Use -2, -1, 0, 1, 2 or 3+.")
  }
  z[z %in% as.character(3:9)] <- "3+"
  z
}
validate_raw <- function(raw, required = candidate_variables) {
  if (!all(required %in% names(raw))) {
    stop(
      "Missing required fields: ", paste(
        setdiff(required, names(raw)),
        collapse = ", "
      )
    )
  }
  for (v in required) {
    x <- raw[[v]]
    if (v %in% status_variables) {
      group_status(x)
    } else {
      if (!is.numeric(x) ||
        anyNA(x) ||
        any(!is.finite(x))) {
```

```
        stop(v, ": enter a finite numerical value.")
      }
      if (v == "AGE" && any(x < 18 | x != floor(x))) {
        stop("AGE: enter a whole number of years, at least 18.")
      }
      if ((v == "LIMIT_BAL" || startsWith(v, "PAY_AMT")) &&
        any(x < 0)) {
        stop(v, ": enter a non-negative amount in NT dollars.")
      }
    }
  }
  invisible(TRUE)
}
prepare_fixed <- function(raw, prep = NULL) {
  validate_raw(raw)
  d <- raw[candidate_variables]
  for (v in money_variables) d[[v]] <- signed_log_amount(d[[v]])
  for (v in status_variables) {
    z <- group_status(d[[v]])
    if (!is.null(prep)) {
      z <- unname(prep$status_mapping[[v]][z])
      d[[v]] <- factor(z, levels = prep$factor_levels[[v]])
    } else {
      d[[v]] <- factor(z, levels = status_levels)
    }
  }
  if ("DEFAULT" %in% names(raw)) {
    d$DEFAULT <- factor(raw$DEFAULT, levels = c("No", "Yes"))
  }
  d
}
fit_preprocessor <- function(raw) {
  d <- prepare_fixed(raw)
  # Pool a rare one-month-delay level into the supported two-month-delay level.
  # Learn this predictor-only rule separately in every training sample.
  support <- lapply(d[status_variables], table)
  mapping_status <- levels_status <- list()
  for (v in status_variables) {
    map <- setNames(status_levels, status_levels)
    if (support[[v]]["1"] < config$rare_status_min) {
      map["1"] <- "2"
    }
    z <- unname(map[as.character(d[[v]])])
    lev <- status_levels[status_levels %in% z]
    if (!all(
      unname(map) %in%
        lev
    )) {
```

```
      stop("Training sample lacks a supported repayment level.")
    }
    mapping_status[[v]] <- map
    levels_status[[v]] <- lev
    d[[v]] <- factor(z, levels = lev)
  }
  f <- stats::reformulate(candidate_variables)
  mm <- stats::model.matrix(f, d)
  terms <- attr(
    stats::terms(f),
    "term.labels"
  )
  mapping <- setNames(
    terms[attr(mm, "assign")[-1]],
    colnames(mm)[-1]
  )
  list(
    formula = f, columns = colnames(mm)[-1],
    term_variable = mapping, required = candidate_variables, status_levels = status_levels,
    status_counts = support, status_mapping = mapping_status, factor_levels = levels_status,
    ranges = lapply(
      raw[setdiff(candidate_variables, status_variables)],
      range
    ),
    transformation = "sign(x)*log1p(abs(x)/1000)", imputation = "None; complete data"
  )
}
design_matrix <- function(prep, d) {
  x <- stats::model.matrix(prep$formula, d)[,
    prep$columns,
    drop = FALSE
  ]
  stopifnot(all(is.finite(x)))
  x
}


# Shared split and nested folds --------------------

make_folds <- function(y, seed, k = 5L) {
  set.seed(seed)
  f <- integer(length(y))
  for (level in levels(y)) {
    rows <- which(y == level)
    f[rows] <- sample(
      rep(
        seq_len(k),
        length.out = length(rows)
      )
    )
  }
  stopifnot(
    all(
      table(f, y) >
        0
    )
  )
  f
}
make_allocations <- function(d) {
  set.seed(config$split_seed)
  dev <- sort(
    unlist(
      lapply(
        split(
          seq_len(nrow(d)),
          d$DEFAULT
        ),
        function(rows) sample(rows, floor(0.8 * length(rows)))
      ),
      use.names = FALSE
    )
  )
  test <- setdiff(
    seq_len(nrow(d)),
    dev
  )
  outer <- make_folds(d$DEFAULT[dev], config$outer_seed)
  inner <- lapply(1:5, function(k) make_folds(d$DEFAULT[dev[outer != k]], config$inner_seeds[k]))
  final_inner <- make_folds(d$DEFAULT[dev], config$final_inner_seed)
  stopifnot(
    !length(intersect(d$ID[dev], d$ID[test])),
    length(dev) +
      length(test) ==
      nrow(d)
  )
  list(
    development = dev, test = test, development_ids = d$ID[dev], test_ids = d$ID[test],
    outer = outer, inner = inner, final_inner = final_inner, seeds = list(
      split = 5L, outer = 1000L, inner = 1001:1005, final = 2001L, bootstrap = 5005L,
      forest = "10000 + stage*1000 + inner_fold; refit=20000 + stage"
```

```
    )
  )
}


# 4. Probability evaluation and calibration --------------------

truth_numeric <- function(y) as.integer(y == "Yes")
check_probability <- function(p, n = length(p)) {
  stopifnot(
    length(p) ==
      n, all(is.finite(p)),
    all(p >= 0 & p <= 1)
  )
  invisible(TRUE)
}
log_loss <- function(y, p) {
  check_probability(p, length(y))
  z <- truth_numeric(y)
  q <- pmax(1e-15, pmin(1 - 1e-15, p))
  -mean(
```

```
    z * log(q) +
      (1 - z) * log1p(-q)
  )
}
roc_object <- function(y, p) {
  pROC::roc(
    y, p,
    levels = c("No", "Yes"),
    direction = "<", quiet = TRUE
  )
}
probability_metrics <- function(y, p, ci = FALSE) {
  check_probability(p, length(y))
  r <- roc_object(y, p)
  a <- c(
    AUC = as.numeric(pROC::auc(r)),
    LogLoss = log_loss(y, p),
    Brier = mean(
      (truth_numeric(y) -
        p)^2
    )
  )
  if (ci) {
    b <- as.numeric(pROC::ci.auc(r, method = "delong"))
    a <- c(a, Lower = b[1], Upper = b[3])
  }
  a
}
threshold_metrics <- function(y, p, threshold = 0.5) {
  z <- truth_numeric(y)
  positive <- p >= threshold
  tp <- sum(z == 1 & positive)
  fp <- sum(z == 0 & positive)
  fn <- sum(z == 1 & !positive)
  tn <- sum(z == 0 & !positive)
  c(
    TP = tp, FP = fp, FN = fn, TN = tn, Sensitivity = tp / (tp + fn), Specificity = tn / (tn +
      fp), PPV = if (tp + fp == 0) NA_real_ else tp / (tp + fp), Accuracy = (tp +
      tn) / length(z),
    Flagged = mean(positive)
  )
}
calibration_bins <- function(y, p, k = 10L) {
  if (!length(y)) {
    return(data.frame())
  }
  # Break prediction ties by stable row order; groups have approximately equal n.
  o <- order(p, seq_along(p))
  g <- ceiling(
    seq_along(p) *
      min(k, length(p)) / length(p)
  )
  do.call(
    rbind, lapply(
      split(o, g),
      function(rows) {
        e <- sum(truth_numeric(y[rows]))
        n <- length(rows)
        ci <- stats::binom.test(e, n)$conf.int
        data.frame(
          n = n, events = e, predicted = mean(p[rows]),
          observed = e / n, lower = ci[1], upper = ci[2]
        )
      }
    )
  )
}
calibration_numbers <- function(y, p) {
  z <- truth_numeric(y)
  lp <- qlogis(pmax(1e-15, pmin(1 - 1e-15, p)))
```

```
  if (length(unique(lp)) <
    2) {
    return(c(Intercept = NA_real_, Slope = NA_real_, OffsetIntercept = NA_real_))
  }
  fit <- glm(z ~ lp, family = binomial())
  offset <- glm(
    z ~ 1 + offset(lp),
    family = binomial()
  )
  c(
```

```
    Intercept = unname(coef(fit)[1]),
    Slope = unname(coef(fit)[2]),
    OffsetIntercept = unname(coef(offset)[1])
  )
}


# 5. Five fitted procedures and prediction --------------------

fit_logistic <- function(d, backward = FALSE) {
  f <- stats::reformulate(candidate_variables, "DEFAULT")
  m <- stats::glm(
    f,
    data = d, family = stats::binomial(), control = stats::glm.control(maxit = 50)
  )
  if (backward) {
    m <- stats::step(m, direction = "backward", k = 2, trace = 0)
  }
  stopifnot(
    isTRUE(m$converged),
    m$rank == length(coef(m)),
    all(is.finite(coef(m)))
  )
  m
}
fit_forest <- function(d, settings, seed) {
  ranger::ranger(
    DEFAULT ~ .,
    data = d, probability = TRUE, num.trees = config$trees, mtry = settings$mtry,
    min.node.size = settings$min.node.size, respect.unordered.factors = "order",
    write.forest = TRUE, seed = seed, num.threads = config$threads, importance = "none"
  )
}
tune_forest <- function(raw, foldid, stage) {
  grid <- config$forest_grid
  scores <- matrix(
    NA_real_, nrow(grid),
    5L
  )
  for (k in 1:5) {
    train <- raw[foldid != k, , drop = FALSE]
    assessment <- raw[foldid == k, , drop = FALSE]
    prep <- fit_preprocessor(train)
    tr <- prepare_fixed(train, prep)
    va <- prepare_fixed(assessment, prep)
    for (j in seq_len(nrow(grid))) {
      m <- fit_forest(tr, grid[j, ], 10000L + stage * 1000L + k)
      p <- predict(m, data = va, num.threads = config$threads)$predictions[
        ,
        "Yes"
      ]
      scores[j, k] <- log_loss(assessment$DEFAULT, p)
      cat(
        sprintf(
          "stage %d forest inner %d grid %d logloss %.6f\n", stage, k, j,
          scores[j, k]
        )
      )
      rm(m)
      gc(verbose = FALSE)
    }
  }
  weights <- as.numeric(table(foldid)) / length(foldid)
  grid$LogLoss <- as.numeric(scores %*% weights)
  best <- order(grid$LogLoss, -grid$min.node.size, grid$mtry)[1]
  list(
    settings = grid[best, c("mtry", "min.node.size")],
    grid = grid, fold_scores = scores, foldid = foldid
  )
}
tune_lasso <- function(raw, foldid, prep, x) {
  full <- glmnet::glmnet(
    x, truth_numeric(raw$DEFAULT),
    family = "binomial", alpha = 1, standardize = TRUE, nlambda = 100, lambda.min.ratio = 1e-04
  )
  grid <- full$lambda
  scores <- matrix(NA_real_, 5, length(grid))
  for (k in 1:5) {
    tr <- raw[foldid != k, ]
    va <- raw[foldid == k, ]
```

```
    pp <- fit_preprocessor(tr)
    xx <- design_matrix(pp, prepare_fixed(tr, pp))
    vx <- design_matrix(pp, prepare_fixed(va, pp))
    m <- glmnet::glmnet(
      xx, truth_numeric(tr$DEFAULT),
      family = "binomial", alpha = 1, standardize = TRUE, lambda = grid
    )
    p <- predict(m, newx = vx, s = grid, type = "response")
    q <- p
    q[] <- pmax(1e-15, pmin(1 - 1e-15, p))
    z <- truth_numeric(va$DEFAULT)
    scores[k, ] <- colMeans(
      -z * log(q) -
        (1 - z) * log1p(-q)
    )
  }
  w <- as.numeric(table(foldid)) / length(foldid)
  means <- colSums(scores * w)
  se <- apply(scores, 2, sd) / sqrt(5)
  minimum <- which.min(means)
  stopifnot(!minimum %in% c(1L, length(grid)))
  chosen <- which(means <= means[minimum] + se[minimum])[1]
  list(
    engine = full, settings = list(
      lambda = grid[chosen], lambda_min = grid[minimum], curve = data.frame(lambda = grid, deviance = 2 * means, SE = 2 * se,
      ↪  terms = full$df),
      fold_scores = scores, foldid = foldid, rule = "Largest lambda within one fold-based SE; SD(fold log loss)/sqrt(5)"
    )
  )
}
fit_family <- function(raw, family, foldid = NULL, stage = 1L, tuning = NULL) {
  prep <- fit_preprocessor(raw)
  d <- prepare_fixed(raw, prep)
  x <- design_matrix(prep, d)
  y <- raw$DEFAULT
  diagnostics <- list(
    n = nrow(raw),
    events = sum(y == "Yes"),
    training_ids_hash = digest::digest(raw$ID),
    status_counts = prep$status_counts, warnings = character()
  )
  fit <- withCallingHandlers(
    {
      if (family %in% c("full", "backward")) {
        engine <- fit_logistic(d, family == "backward")
        required <- attr(
          stats::terms(engine),
          "term.labels"
        )
        selected <- names(coef(engine))[-1]
        diagnostics$converged <- engine$converged
        diagnostics$rank <- engine$rank
        diagnostics$max_abs_coefficient <- max(abs(coef(engine)))
        diagnostics$probability_range <- range(fitted(engine))
        diagnostics$boundary_probabilities <- sum(
          fitted(engine) <
            1e-08 | fitted(engine) >
            1 - 1e-08
        )
        diagnostics$AIC <- AIC(engine)
        settings <- list(
          formula = paste(
            deparse(formula(engine)),
            collapse = " "
          )
        )
      } else if (family == "lda") {
        s <- apply(x, 2, sd)
        cols <- names(s)[s >
          1e-10]
        q <- qr(scale(x[, cols, drop = FALSE], center = TRUE, scale = FALSE))
        cols <- cols[q$pivot[seq_len(q$rank)]]
        prep$lda_columns <- cols
        prep$center <- colMeans(x[, cols, drop = FALSE])
        prep$scale <- s[cols]
        sx <- scale(x[, cols, drop = FALSE], prep$center, prep$scale)
        engine <- MASS::lda(sx, grouping = y, prior = as.numeric(table(y)) / length(y))
        required <- unique(unname(prep$term_variable[cols]))
        selected <- cols
        settings <- list(prior = engine$prior)
        diagnostics$excluded_design_columns <- setdiff(
```

```
          colnames(x),
          cols
        )
      } else if (family == "lasso") {
        stopifnot(
          length(foldid) ==
            nrow(raw)
        )
        tuned <- tune_lasso(raw, foldid, prep, x)
        engine <- tuned$engine
```

```
        settings <- tuned$settings
        b <- as.matrix(coef(engine, s = settings$lambda))[
          ,
          1
        ]
        selected <- setdiff(
          names(b)[b !=
            0], "(Intercept)"
        )
        required <- unique(unname(prep$term_variable[selected]))
        settings$coefficients <- b
        diagnostics$lambda_min_at_boundary <- FALSE
        eligible <- which(
          settings$curve$deviance <= min(settings$curve$deviance) +
            settings$curve$SE[which.min(settings$curve$deviance)]
        )
        stopifnot(
          isTRUE(
            all.equal(
              max(settings$curve$lambda[eligible]),
              settings$lambda
            )
          )
        )
      } else if (family == "forest") {
        if (is.null(tuning)) {
          tuning <- tune_forest(raw, foldid, stage)
        }
        engine <- fit_forest(d, tuning$settings, 20000L + stage)
        required <- candidate_variables
        selected <- candidate_variables
        settings <- c(
          as.list(tuning$settings),
          list(trees = config$trees, tuning = tuning)
        )
        diagnostics$OOB_Brier <- engine$prediction.error
      } else {
        stop("Unknown family")
      }
      list(
        family = family, engine = engine, prep = prep, required = required,
        selected_terms = selected, settings = settings, outcome_levels = c("No", "Yes")
      )
    },
    warning = function(w) {
      diagnostics$warnings <<- c(diagnostics$warnings, conditionMessage(w))
      # Record locally and in the execution log; never suppress warnings globally.
    }
  )
  fit$diagnostics <- diagnostics
  fit
}


# Frozen raw-profile prediction --------------------

predict_raw <- function(bundle, raw) {
  validate_raw(raw, bundle$required)
  # Fill unused fields deterministically, so they never affect validation/prediction.
  d <- raw
  unused <- setdiff(candidate_variables, bundle$required)
  for (v in unused) {
    d[[v]] <- if (v %in% status_variables) {
      "-1"
    } else if (v == "AGE") {
      30
    } else {
      0
    }
  }
```

```
  d <- prepare_fixed(d, bundle$prep)
  if (bundle$family %in% c("full", "backward")) {
    p <- stats::predict(bundle$engine, newdata = d, type = "response")
  } else if (bundle$family == "lda") {
    x <- design_matrix(bundle$prep, d)[,
      bundle$prep$lda_columns,
      drop = FALSE
    ]
    x <- scale(x, bundle$prep$center, bundle$prep$scale)
    p <- predict(bundle$engine, newdata = x)$posterior[
      ,
      "Yes"
    ]
  } else if (bundle$family == "lasso") {
    p <- predict(
      bundle$engine,
      newx = design_matrix(bundle$prep, d),
      s = bundle$settings$lambda, type = "response"
    )
  } else if (bundle$family == "forest") {
    p <- predict(bundle$engine, data = d, num.threads = 1L)$predictions[
```

```
      ,
      "Yes"
    ]
  } else {
    stop("Unsupported saved model family")
  }
  p <- as.numeric(p)
  check_probability(p, nrow(raw))
  p
}


# 6. Backward procedure bootstrap and held-out importance --------------------

bootstrap_selection <- function(dev, manifest, indices, B = 200L) {
  dir.create("results/bootstrap", showWarnings = FALSE)
  apparent <- fit_family(dev, "backward")
  auc_app <- as.numeric(pROC::auc(roc_object(dev$DEFAULT, predict_raw(apparent, dev))))
  start <- proc.time()[3]
  out <- vector("list", B)
  for (b in seq_len(B)) {
    path <- sprintf("results/bootstrap/rep_%03d.rds", b)
    if (file.exists(path)) {
      result <- readRDS(path)
      assert_manifest(result$manifest, manifest$data_hash)
    } else {
      result <- tryCatch(
        {
          boot <- dev[indices[, b], , drop = FALSE]
          m <- fit_family(boot, "backward")
          ab <- as.numeric(pROC::auc(roc_object(boot$DEFAULT, predict_raw(m, boot))))
          ao <- as.numeric(pROC::auc(roc_object(dev$DEFAULT, predict_raw(m, dev))))
          list(
            auc_bootstrap = ab, auc_original = ao, optimism = ab - ao, selected = m$required,
            selected_terms = m$selected_terms, diagnostics = m$diagnostics,
            error = NULL
          )
        },
        error = function(e) list(error = conditionMessage(e))
      )
      result$manifest <- manifest
      result$replicate <- b
      save_atomic(result, path)
    }
    out[[b]] <- result
    if (b %% 5 == 0) {
      cat(
        sprintf(
          "bootstrap %d/%d elapsed %.1f minutes\n", b, B, (proc.time()[3] -
            start) / 60
        )
      )
    }
    if (b == 20) {
      cat(
        sprintf(
          "20-replicate pilot complete; projected total %.1f minutes\n",
          (proc.time()[3] - start) / 60 * B / 20
        )
```

```
      )
    }
  }
  success <- vapply(
    out, function(x) is.null(x$error),
    logical(1)
  )
  if (!all(success)) {
    stop("Bootstrap fit failures saved; diagnose before accepting summaries.")
  }
  optimism <- vapply(
    out, `[[`, numeric(1),
    "optimism"
  )
  inclusion <- vapply(
    candidate_variables, function(v) {
      mean(
        vapply(
          out, function(x) {
            v %in%
              x$selected
          }, logical(1)
        )
      )
    },
    numeric(1)
  )
  result <- list(
    B = B, successful = sum(success),
    failures = sum(!success),
    apparent_AUC = auc_app, mean_optimism = mean(optimism),
    corrected_AUC = auc_app - mean(optimism),
```

```
    optimism_MCSE = sd(optimism) / sqrt(B),
    inclusion = inclusion, replicates = out, manifest = manifest
  )
  save_atomic(result, "results/bootstrap_summary.rds")
  utils::write.csv(
    data.frame(
      variable = names(inclusion),
      frequency = inclusion, MCSE = sqrt(inclusion * (1 - inclusion) / B)
    ),
    "results/bootstrap_stability.csv",
    row.names = FALSE
  )
  result
}


# Cross-validated permutation importance --------------------

heldout_importance <- function(dev, alloc, chosen) {
  scores <- list()
  i <- 0L
  for (k in 1:5) {
    m <- readRDS(sprintf("results/outer/fold_%d.rds", k))$models[[chosen]]
    d <- dev[alloc$outer == k, , drop = FALSE]
    base <- log_loss(d$DEFAULT, predict_raw(m, d))
    for (v in candidate_variables) {
      for (r in 1:3) {
        set.seed(
          70000L + k * 1000L + match(v, candidate_variables) *
            10L + r
        )
        perm <- d
        perm[[v]] <- sample(perm[[v]])
        i <- i + 1L
        scores[[i]] <- data.frame(
          fold = k, variable = v, repeat_id = r, n = nrow(d),
          increase = log_loss(d$DEFAULT, predict_raw(m, perm)) -
            base
        )
      }
    }
  }
  long <- do.call(rbind, scores)
  summary <- do.call(
    rbind, lapply(
      candidate_variables, function(v) {
        z <- long[long$variable == v, ]
        data.frame(variable = v, increase = weighted.mean(z$increase, z$n))
```

```
      }
    )
  )
  summary <- summary[order(-summary$increase), ]
  utils::write.csv(long, "results/permutation_detail.csv", row.names = FALSE)
  utils::write.csv(summary, "results/permutation_importance.csv", row.names = FALSE)
  summary
}


# Offline analysis stages (explicit rebuild only) ----------------------------

run_audit <- function() {
  input <- read_credit()
  a <- audit_credit(input)
  allocation <- make_allocations(input$raw)
  manifest <- stage_manifest(input$hash)
  save_atomic(input$raw[allocation$development, ], "results/development_data.rds")
  save_atomic(input$raw[allocation$test, ], "results/locked_test_data.rds")
  save_atomic(
    list(audit = a, allocation = allocation, manifest = manifest),
    "results/audit_bundle.rds"
  )
  utils::write.csv(
    data.frame(
      ID = input$raw$ID, sample = ifelse(
        seq_len(nrow(input$raw)) %in%
          allocation$development, "development", "test"
      )
    ),
    "results/split_ids.csv",
    row.names = FALSE
  )
  save_atomic(allocation, "results/resampling.rds")
  write_json(allocation$seeds, "results/seeds.json")
  d <- input$raw[allocation$development, ]
  describe_development(d)
  prep <- fit_preprocessor(d)
  save_atomic(prep, "results/preprocessing_schema.rds")
  for (family in c("full", "lda")) {
    m <- fit_family(d, family)
    check_probability(
      predict_raw(m, d),
```

```
      nrow(d)
    )
    save_atomic(m, paste0("results/", family, "_baseline.rds"))
    cat(family, "foundation fit passed\n")
  }
  print(
    a[c("rows", "columns", "events", "missing", "unique_ids", "duplicate_profiles")]
  )
  cat("AUDIT AND FOUNDATION COMPLETE\n")
}
run_develop <- function() {
  a <- read_audit()
  alloc <- a$allocation
  manifest <- a$manifest
  dev <- readRDS("results/development_data.rds")
  stopifnot(
    identical(dev$ID, alloc$development_ids),
    !any(dev$ID %in% alloc$test_ids)
  )
  outer <- vector("list", 5)
  for (k in 1:5) {
    path <- sprintf("results/outer/fold_%d.rds", k)
    if (file.exists(path)) {
      out <- readRDS(path)
      assert_manifest(out$manifest, manifest$data_hash)
      cat("Reused outer fold", k, "\n")
    } else {
      tr <- dev[alloc$outer != k, ]
      va <- dev[alloc$outer == k, ]
      models <- list()
      probabilities <- data.frame(ID = va$ID, DEFAULT = va$DEFAULT, fold = k)
      for (f in families) {
        cat("Outer fold", k, "fit", f, "\n")
        models[[f]] <- fit_family(tr, f, alloc$inner[[k]], k)
        probabilities[[f]] <- predict_raw(models[[f]], va)
      }
      probabilities$baseline <- mean(tr$DEFAULT == "Yes")
```

```
      out <- list(
        models = models, predictions = probabilities, manifest = manifest,
        training_ids = tr$ID, assessment_ids = va$ID, inner_foldid = alloc$inner[[k]]
      )
      save_atomic(out, path)
    }
    outer[[k]] <- out
  }
  oof <- do.call(rbind, lapply(outer, `[[`, "predictions"))
  oof <- oof[match(dev$ID, oof$ID), ]
  stopifnot(
    identical(oof$ID, dev$ID),
    !anyDuplicated(oof$ID),
    !anyNA(oof),
    nrow(oof) ==
      nrow(dev)
  )
  metrics <- do.call(
    rbind, lapply(
      c(families, "baseline"),
      function(f) data.frame(family = f, as.list(probability_metrics(oof$DEFAULT, oof[[f]])))
    )
  )
  fold_metrics <- do.call(
    rbind, lapply(
      1:5, function(k) {
        do.call(
          rbind, lapply(
            c(families, "baseline"),
            function(f) {
              z <- oof[oof$fold == k, ]
              data.frame(
                fold = k, family = f, n = nrow(z),
                as.list(probability_metrics(z$DEFAULT, z[[f]]))
              )
            }
          )
        )
      }
    )
  )
  utils::write.csv(metrics, "results/cv_metrics.csv", row.names = FALSE)
  utils::write.csv(fold_metrics, "results/cv_fold_metrics.csv", row.names = FALSE)
  save_atomic(oof, "results/oof_predictions.rds")
  cat("Shared nested CV complete\n")
  print(metrics)
  final <- list()
  for (f in families) {
    path <- paste0("results/final/", f, ".rds")
    if (file.exists(path)) {
      m <- readRDS(path)
      assert_manifest(m$manifest, manifest$data_hash)
    } else {
      cat("Complete development fit", f, "\n")
```

```
    tuning <- NULL
    if (f == "forest" && file.exists("results/final/forest_tuning.rds")) {
      cache <- readRDS("results/final/forest_tuning.rds")
      assert_manifest(cache$manifest, manifest$data_hash)
      tuning <- cache$tuning
    }
    m <- fit_family(dev, f, alloc$final_inner, 6L, tuning = tuning)
    m$manifest <- manifest
    save_atomic(m, path)
  }
  final[[f]] <- m
}
indices_path <- "results/bootstrap_indices.rds"
if (file.exists(indices_path)) {
  indices <- readRDS(indices_path)
} else {
  set.seed(config$bootstrap_seed)
  indices <- replicate(
    config$bootstrap_B, sample.int(
      nrow(dev),
      nrow(dev),
      replace = TRUE
    )
  )
  save_atomic(indices, indices_path)
}
```

```
boot <- bootstrap_selection(dev, manifest, indices, config$bootstrap_B)
primary <- metrics[metrics$family %in% families, ]
best <- primary$family[which.min(primary$LogLoss)]
best_row <- primary[primary$family == best, ]
primary$inputs <- vapply(
  final, function(m) length(m$required),
  integer(1)
)[primary$family]
primary$eligible <- primary$LogLoss <= config$logloss_tolerance * best_row$LogLoss &
  primary$AUC >= best_row$AUC - config$auc_tolerance
eligible <- primary[primary$eligible, ]
order_family <- c("lasso", "backward", "full", "lda", "forest")
chosen <- eligible$family[order(eligible$inputs, eligible$LogLoss, match(eligible$family, order_family))[1]]
rival <- primary$family[order(primary$LogLoss)][which(
  primary$family[order(primary$LogLoss)] !=
    chosen
)[1]]
calibration <- do.call(
  rbind, lapply(
    families, function(f) data.frame(family = f, as.list(calibration_numbers(oof$DEFAULT, oof[[f]])))
  )
)
utils::write.csv(calibration, "results/cv_calibration.csv", row.names = FALSE)
lock <- list(
  chosen = chosen, rival = rival, best_logloss = best, metrics = primary, rule = "Within 1.01 * minimum log loss and AUC >= best
  ↪  AUC - .01; fewest original inputs; log loss; fixed family order",
  required = final[[chosen]]$required, settings = final[[chosen]]$settings,
  development_ids_hash = digest::digest(dev$ID),
  manifest = manifest, models_hash = setNames(
    vapply(
      paste0("results/final/", families, ".rds"),
      digest::digest, character(1),
      file = TRUE, algo = "sha256"
    ),
    families
  ),
  locked_at = format(Sys.time(), tz = "Europe/Amsterdam", usetz = TRUE)
)
if (file.exists("results/model_selection.rds")) {
  old <- readRDS("results/model_selection.rds")
  stopifnot(
    identical(old$chosen, chosen),
    identical(old$models_hash, lock$models_hash)
  )
  lock <- old
} else {
  save_atomic(lock, "results/model_selection.rds")
  write_json(lock, "results/model_selection.json")
}
importance <- heldout_importance(dev, alloc, chosen)
# Model coefficients and factor joint tests are descriptive development diagnostics.
utils::write.csv(
  data.frame(
    term = names(coef(final$full$engine)),
    coefficient = coef(final$full$engine)
  ),
  "results/full_coefficients.csv",
  row.names = FALSE
)
joint <- drop1(final$full$engine, test = "Chisq")
utils::write.csv(
  data.frame(
    term = rownames(joint),
    joint
  ),
```

```
    "results/full_joint_tests.csv",
    row.names = FALSE
  )
  effects <- do.call(
    rbind, lapply(
      setdiff(candidate_variables, status_variables),
      function(v) {
        q <- quantile(dev[[v]], c(0.25, 0.75))
        b <- coef(final$full$engine)[v]
        delta <- if (v %in% money_variables) {
          diff(signed_log_amount(q))
        } else {
          diff(q)
        }
        data.frame(
          variable = v, Q1 = q[1], Q3 = q[2], OR_Q3_vs_Q1 = unname(exp(b * delta)),
```

```
          row.names = NULL
        )
      }
    )
  )
  utils::write.csv(effects, "results/full_IQR_effects.csv", row.names = FALSE)
  cat("DEVELOPMENT COMPLETE; SELECTION LOCK:", chosen, "rival:", rival, "\n")
  print(primary)
  print(boot[c("B", "failures", "apparent_AUC", "mean_optimism", "corrected_AUC")])
}
run_evaluate <- function() {
  a <- read_audit()
  if (!file.exists("results/model_selection.rds")) {
    stop("Test evaluation requires selection lock.")
  }
  lock <- readRDS("results/model_selection.rds")
  assert_manifest(lock$manifest, a$manifest$data_hash)
  actual <- setNames(
    vapply(
      paste0("results/final/", families, ".rds"),
      digest::digest, character(1),
      file = TRUE, algo = "sha256"
    ),
    families
  )
  stopifnot(identical(actual, lock$models_hash))
  if (file.exists("results/test_evaluation.rds")) {
    old <- readRDS("results/test_evaluation.rds")
    assert_manifest(old$manifest, a$manifest$data_hash)
    cat("Reused locked evaluation; no refitting\n")
    return(invisible(old))
  }
  test <- readRDS("results/locked_test_data.rds")
  stopifnot(identical(test$ID, a$allocation$test_ids))
  pred <- data.frame(ID = test$ID, DEFAULT = test$DEFAULT)
  for (f in families) {
    pred[[f]] <- predict_raw(
      readRDS(paste0("results/final/", f, ".rds")),
      test
    )
  }
  pred$baseline <- a$audit$events * 0 + mean(
    readRDS("results/development_data.rds")$DEFAULT ==
      "Yes"
  )
  metrics <- do.call(
    rbind, lapply(
      c(families, "baseline"),
      function(f) data.frame(family = f, as.list(probability_metrics(test$DEFAULT, pred[[f]], ci = TRUE)))
    )
  )
  thresholds <- do.call(
    rbind, lapply(
      c(families, "baseline"),
      function(f) data.frame(family = f, as.list(threshold_metrics(test$DEFAULT, pred[[f]], 0.5)))
    )
  )
  calibration <- setNames(
    lapply(
      c(families, "baseline"),
      function(f) calibration_bins(test$DEFAULT, pred[[f]])
    ),
    c(families, "baseline")
  )
  cal_numbers <- do.call(
    rbind, lapply(
      c(families, "baseline"),
      function(f) data.frame(family = f, as.list(calibration_numbers(test$DEFAULT, pred[[f]])))
    )
  )
  paired <- pROC::roc.test(
    roc_object(test$DEFAULT, pred[[lock$chosen]]),
    roc_object(test$DEFAULT, pred[[lock$rival]]),
    method = "delong", paired = TRUE, conf.level = 0.95
  )
```

```
  comparison <- list(
    selected = lock$chosen, rival = lock$rival, difference = unname(paired$estimate[1] - paired$estimate[2]),
    CI = as.numeric(paired$conf.int),
    p = paired$p.value
  )
```

```
  evaluation <- list(
    predictions = pred, metrics = metrics, thresholds = thresholds, calibration = calibration,
    calibration_numbers = cal_numbers, paired = comparison, manifest = a$manifest,
    selection_lock_hash = digest::digest("results/model_selection.rds", file = TRUE, algo = "sha256"),
    evaluated_at = format(Sys.time(), tz = "Europe/Amsterdam", usetz = TRUE)
  )
  save_atomic(evaluation, "results/test_evaluation.rds")
  utils::write.csv(metrics, "results/test_metrics.csv", row.names = FALSE)
  utils::write.csv(thresholds, "results/test_threshold_050.csv", row.names = FALSE)
  utils::write.csv(cal_numbers, "results/test_calibration.csv", row.names = FALSE)
  write_json(comparison, "results/paired_comparison.json")
  cat("LOCKED TEST EVALUATION COMPLETE\n")
  print(metrics)
  print(comparison)
}


summarize_analysis <- function() {
  a <- read_audit()
  models <- setNames(
    lapply(families, function(f) readRDS(paste0("results/final/", f, ".rds"))),
    families
  )
  for (f in families) assert_manifest(models[[f]]$manifest, data_hash())
  specifications <- do.call(
    rbind, lapply(
      families, function(f) {
        m <- models[[f]]
        data.frame(
          family = f, original_inputs = length(m$required),
          terms = length(m$selected_terms),
          retained = paste(m$required, collapse = ", "),
          specification = if (f %in% c("full", "backward")) {
            m$settings$formula
          } else if (f == "lasso") {
            paste0("lambda.1se=", format(m$settings$lambda, digits = 8))
          } else if (f == "forest") {
            paste0(
              "mtry=", m$settings$mtry, "; min.node.size=", m$settings$min.node.size,
              "; trees=", m$settings$trees
            )
          } else {
            "Empirical class priors; training-scaled full-rank dummy matrix"
          }
        )
      }
    )
  )
  utils::write.csv(specifications, "results/development_specifications.csv", row.names = FALSE)
  l <- models$lasso
  curve <- l$settings$curve
  rows <- match(
    c(l$settings$lambda, l$settings$lambda_min),
    curve$lambda
  )
  sensitivity <- data.frame(
    rule = c("lambda.1se (primary)", "lambda.min (development sensitivity)"),
    curve[rows, ]
  )
  sensitivity$inputs <- vapply(
    c(l$settings$lambda, l$settings$lambda_min),
    function(s) {
      b <- as.matrix(coef(l$engine, s = s))[
        ,
        1
      ]
      selected <- setdiff(
        names(b)[b !=
          0], "(Intercept)"
      )
      length(unique(unname(l$prep$term_variable[selected])))
    }, integer(1)
  )
  utils::write.csv(sensitivity, "results/lasso_development_sensitivity.csv", row.names = FALSE)
  utils::write.csv(
    data.frame(
      term = names(l$settings$coefficients),
      coefficient = l$settings$coefficients, original_variable = c("Intercept",
      ↪  unname(l$prep$term_variable[names(l$settings$coefficients)[-1]]))
    ),
    "results/lasso_coefficients.csv",
```

```
    row.names = FALSE
  )
  utils::write.csv(
    models$forest$settings$tuning$grid, "results/forest_final_tuning.csv",
    row.names = FALSE
  )
  diagnostics <- lapply(models, function(m) m$diagnostics)
  save_atomic(diagnostics, "results/development_fit_diagnostics.rds")
  diagnostics_json <- lapply(
    diagnostics, function(x) {
      x$status_counts <- lapply(
        x$status_counts, function(t) {
          setNames(
            as.list(as.integer(t)),
            names(t)
          )
        }
      )
      x
    }
  )
  write_json(diagnostics_json, "results/development_fit_diagnostics.json")
  oof <- readRDS("results/oof_predictions.rds")
  bins <- do.call(
    rbind, lapply(
      families, function(f) data.frame(family = f, calibration_bins(oof$DEFAULT, oof[[f]]))
    )
  )
  utils::write.csv(bins, "results/cv_calibration_bins.csv", row.names = FALSE)
  cat("DEVELOPMENT SPECIFICATIONS, TUNING, SENSITIVITY AND CALIBRATION SAVED\n")
  print(specifications[, 1:3])
  print(sensitivity)
}

data_hash <- function() {
  path <- list.files("data", pattern = "[.]csv$", full.names = TRUE, ignore.case = TRUE)
  stopifnot(
    length(path) ==
      1L
  )
  sha256_file(path)
}

read_audit <- function() {
  audit <- readRDS("results/audit_bundle.rds")
  assert_manifest(audit$manifest, data_hash())
  audit
}

rebuild_analysis <- function(data_file, work_dir = "analysis_work") {
  for (package in c("MASS", "glmnet", "ranger", "pROC", "digest", "jsonlite", "ggplot2", "readxl")) {
    if (!requireNamespace(package, quietly = TRUE)) {
      stop("Install this analysis prerequisite: ", package)
    }
  }
  stopifnot(file.exists(data_file))
  input <- normalizePath(data_file, winslash = "/", mustWork = TRUE)
  dir.create(work_dir, recursive = TRUE, showWarnings = FALSE)
  work_dir <- normalizePath(work_dir, winslash = "/", mustWork = TRUE)
  for (directory in c("data", "results", "logs", "results/outer", "results/final", "results/figures")) {
    dir.create(
      file.path(work_dir, directory),
      recursive = TRUE, showWarnings = FALSE
    )
  }
  destination <- file.path(work_dir, "data", "UCI_Credit_Card.csv")
  if (!identical(input, destination)) {
    if (file.exists(destination)) {
      stopifnot(
        identical(
          sha256_file(input),
          sha256_file(destination)
        )
      )
    } else {
      stopifnot(file.copy(input, destination))
    }
  }
  source_copy <- file.path(work_dir, "CreditLens.R")
  if (!identical(source_copy, .creditlens_file)) {
```

```
    stopifnot(file.copy(.creditlens_file, source_copy, overwrite = TRUE))
  }
  previous <- getwd()
  on.exit(
    setwd(previous),
    add = TRUE
  )
  setwd(work_dir)
  if (file.exists("results/audit_bundle.rds")) {
    read_audit()
  } else {
```

```
    run_audit()
  }
  run_develop()
  run_evaluate()
  summarize_analysis()
  verify_original_workbook(destination)
  cache <- list(
    data_hash = data_hash(), sources = c(CreditLens.R = sha256_file(source_copy)),
    completion = "Development complete and choice locked", package_versions = stage_manifest(data_hash())$packages,
    R = R.version.string
  )
  save_atomic(cache, "results/analysis_cache_manifest.rds")
  invisible(work_dir)
}

verify_original_workbook <- function(csv_path) {
  directory <- tempfile("creditlens-provenance-")
  dir.create(directory)
  on.exit(
    unlink(directory, recursive = TRUE),
    add = TRUE
  )
  url <- "https://archive.ics.uci.edu/static/public/350/default%2Bof%2Bcredit%2Bcard%2Bclients.zip"
  archive <- file.path(directory, "original.zip")
  utils::download.file(url, archive, mode = "wb", quiet = TRUE)
  files <- utils::unzip(archive, exdir = directory)
  workbook <- files[grepl("[.]xls$", files, ignore.case = TRUE)]
  stopifnot(
    length(workbook) ==
      1L
  )
  original <- as.data.frame(readxl::read_excel(workbook, skip = 1))
  local <- utils::read.csv(csv_path, check.names = FALSE)
  stopifnot(
    identical(
      dim(original),
      dim(local)
    ),
    all(
      as.matrix(original) ==
        as.matrix(local)
    )
  )
  write_json(
    list(
      source_url = url, checked = as.character(Sys.Date()),
      rows = nrow(local),
      columns = ncol(local),
      all_750000_values_equal = TRUE, uci_zip_sha256 = sha256_file(archive),
      local_csv_sha256 = sha256_file(csv_path)
    ),
    "results/provenance_comparison.json"
  )
}

# 7. Release identity, dependencies and inspection --------------------------

sha256_file <- function(path) {
  digest::digest(path, file = TRUE, algo = "sha256")
}

function_signature <- function(fun) {
  # Discard formatting and source-reference attributes, preserving expressions.
  normalize <- function(expression) {
    if (is.call(expression)) {
      if (identical(expression[[1]], as.name("{")) &&
        length(expression) ==
          2L) {
        return(normalize(expression[[2]]))
      }
      return(
        as.call(
          lapply(
            as.list(expression),
            normalize
          )
        )
      )
    }
    expression
  }
  paste(
    deparse(
      list(
        formals(fun),
        normalize(body(fun))
      ),
      width.cutoff = 500L
    ),
    collapse = "\n"
  )
```

```
}

analysis_function_names <- c(
  "signed_log_amount", "group_status", "validate_raw", "prepare_fixed", "fit_preprocessor",
  "design_matrix", "make_folds", "make_allocations", "truth_numeric", "check_probability",
  "log_loss", "roc_object", "probability_metrics", "threshold_metrics", "calibration_bins",
  "calibration_numbers", "fit_logistic", "fit_forest", "tune_forest", "tune_lasso",
  "fit_family", "predict_raw", "bootstrap_selection", "heldout_importance"
)

code_fingerprint <- function() {
  signatures <- vapply(
    analysis_function_names, function(name) {
      function_signature(get(name, mode = "function"))
    }, character(1)
  )
  digest::digest(
    list(config = config, functions = signatures),
    algo = "sha256"
  )
}

stage_manifest <- function(data_hash) {
  packages <- c("MASS", "glmnet", "ranger", "pROC", "digest")
  list(
    data_hash = data_hash, code_hash = code_fingerprint(), config = config, R = R.version.string,
    packages = vapply(
      packages, function(package) {
        as.character(utils::packageVersion(package))
      }, character(1)
    )
  )
}

assert_manifest <- function(manifest, data_hash) {
  stopifnot(
    identical(manifest$data_hash, data_hash),
    identical(manifest$code_hash, code_fingerprint()),
    identical(manifest$config, config),
    identical(manifest$packages, stage_manifest(data_hash)$packages)
  )
  invisible(TRUE)
}

tested_dependencies <- function() {
  packages <- c(
    "shiny", "bslib", "htmltools", "ggplot2", "glmnet", "Matrix", "pROC", "digest",
    "base64enc", "MASS", "ranger", "jsonlite", "readxl", "rmarkdown", "knitr",
    "rsconnect"
  )
  data.frame(
    package = c("R", packages),
    version = c(
      as.character(getRversion()),
      vapply(
        packages, function(package) {
          if (requireNamespace(package, quietly = TRUE)) {
            as.character(utils::packageVersion(package))
```

```
          } else {
            "Not installed"
          }
        }, character(1)
      )
    )
  )
}

model_identity <- function(model) {
  # The identity covers the actual selected parameters and fitted preparation.
  preparation <- model$prep
  environment(preparation$formula) <- baseenv()
  digest::digest(
    list(
      family = model$family, preparation = preparation, inputs = model$required,
      lambda = model$settings$lambda, coefficients = model$settings$coefficients
    ),
    algo = "sha256"
  )
}

asset_checksum <- function(assets) {
  assets$integrity <- NULL
  digest::digest(assets, algo = "sha256")
}

load_creditlens_assets <- function(root = .creditlens_root, verify_source = TRUE) {
  for (package in c("glmnet", "pROC", "digest")) {
    if (!requireNamespace(package, quietly = TRUE)) {
      stop("Install the required package before running CreditLens: ", package)
    }
  }
```

```
  path <- file.path(root, "creditlens_assets.rds")
  if (!file.exists(path)) {
    stop("Keep creditlens_assets.rds beside CreditLens.R.")
  }
  assets <- readRDS(path)
  stopifnot(
    identical(assets$integrity, asset_checksum(assets)),
    identical(assets$identity$model_id, model_identity(assets$app$runtime$model)),
    identical(assets$app$runtime$identity, assets$identity),
    identical(assets$report$identity, assets$identity)
  )
  if (verify_source) {
    stopifnot(identical(assets$source_sha256, sha256_file(file.path(root, "CreditLens.R"))))
    report <- file.path(root, "individual_assignment.Rmd")
    if (file.exists(report)) {
      stopifnot(identical(assets$report_sha256, sha256_file(report)))
    }
  }
  assets
}

inspect_creditlens <- function(component = NULL, root = .creditlens_root) {
  assets <- load_creditlens_assets(root)
  records <- c(
    list(
      identity = assets$identity, model = assets$app$runtime$model, coefficients = assets$coefficients,
      dependencies = assets$dependencies, deployment = assets$deployment, verification = assets$verification
    ),
    assets$evidence
  )
  if (is.null(component)) {
    return(names(records))
  }
  if (!component %in% names(records)) {
    stop("Choose a component from inspect_creditlens().")
  }
  records[[component]]
}

final_coefficient_table <- function(model) {
  coefficients <- model$settings$coefficients
  retained <- coefficients != 0 | names(coefficients) ==
    "(Intercept)"
  terms <- names(coefficients)[retained]
  variables <- unname(model$prep$term_variable[terms])
  variables[terms == "(Intercept)"] <- "Intercept"
  interpretation <- vapply(
```

```
    seq_along(terms),
    function(index) {
      variable <- variables[index]
      if (variable == "Intercept") {
        return("Intercept on the transformed design scale")
      }
      if (variable %in% status_variables) {
        code <- substring(
          terms[index], nchar(variable) +
            1L
        )
        if (code == "2" && model$prep$status_mapping[[variable]]["1"] ==
          "2") {
          code <- "1-2 (pooled)"
        }
        return(paste0("Category ", code, "; reference -1 (paid in full)"))
      }
      if (variable == "AGE") {
        return("Per additional year, holding the other inputs fixed")
      }
      "Per unit of sign(x) * log(1 + abs(x)/1000), x in NT dollars"
    }, character(1)
  )
  data.frame(
    term = terms, original_input = variables, coefficient = unname(coefficients[retained]),
    interpretation = interpretation, row.names = NULL
  )
}

repayment_metadata <- function(model) {
  list(
    labels = c(
      `-2` = "No consumption", `-1` = "Paid in full (UCI: paid duly)", `0` = "Use of revolving credit",
      `1` = "One month delayed", `2` = "Two months delayed", `3+` = "Three or more months delayed"
    ),
    explanation = paste(
      "Expanded published coding supplements the abbreviated UCI description.",
      "Codes -2, -1 and 0 remain separate nominal categories.", "Codes 3-9 form 3+. Within each training sample, code 1 is pooled
      ↪  into",
      "code 2 when fewer than 50 observations have code 1.", "The complete-development fit pools August-April; September stays
      ↪  distinct."
    ),
    references = c(
      "Chen and Ye (2022), section 4.1.1: https://arxiv.org/pdf/2209.10070",
```

```
      "H2O.ai: https://h2oai.github.io/tutorials/machine-learning-interpretability/"
    ),
    fitted_mapping = model$prep$status_mapping, training_counts = model$prep$status_counts
  )
}

# 8. Verification of frozen assets and historical analysis ------------------

verify_release <- function(root = .creditlens_root, assets = NULL) {
  if (is.null(assets)) {
    assets <- load_creditlens_assets(root)
  }
  checks <- character()
  passed <- function(label) {
    checks <<- c(checks, label)
  }
  expect_error <- function(expression) {
    stopifnot(
      inherits(
        tryCatch(
          force(expression),
          error = identity
        ),
        "error"
      )
    )
  }
  runtime <- assets$app$runtime
  model <- runtime$model
  evidence <- assets$evidence
  stopifnot(
    identical(assets$integrity, asset_checksum(assets)),
    identical(model$family, "lasso"),
    length(model$required) ==
      14L, length(model$selected_terms) ==
      18L, identical(assets$identity$model_id, model_identity(model)),
    identical(assets$report$identity, runtime$identity)
  )
```

```
  passed("Asset checksum, shared app/report release and frozen model identity")

  # Independently recalculate the prespecified choice from development scores.
  selection <- evidence$selection_lock
  scores <- selection$metrics
  best <- scores[which.min(scores$LogLoss), ]
  eligible <- scores[scores$LogLoss <= 1.01 * best$LogLoss & scores$AUC >= best$AUC -
    0.01, ]
  ordering <- order(
    eligible$inputs, eligible$LogLoss, match(eligible$family, c("lasso", "backward", "full", "lda", "forest"))
  )
  stopifnot(identical(eligible$family[ordering[1]], selection$chosen))
  tuning <- evidence$tuning$lasso
  curve <- tuning$curve
  minimum <- which.min(curve$deviance)
  qualifying <- curve$lambda[curve$deviance <= curve$deviance[minimum] + curve$SE[minimum]]
  stopifnot(
    identical(
      max(qualifying),
      tuning$lambda
    )
  )
  stopifnot(
    as.POSIXct(selection$locked_at) <
      as.POSIXct(evidence$holdout$evaluated_at),
    identical(evidence$holdout$selection_lock_hash, evidence$selection_lock_sha256)
  )
  passed("Development family rule, one-SE penalty and lock preceding holdout")

  allocation <- evidence$allocations
  stopifnot(
    !length(intersect(allocation$development_ids, allocation$test_ids)),
    length(allocation$outer) ==
      runtime$development_n, length(allocation$final_inner) ==
      runtime$development_n, identical(evidence$cv_predictions$ID, allocation$development_ids),
    !anyDuplicated(evidence$cv_predictions$ID),
    identical(evidence$holdout$predictions$ID, allocation$test_ids)
  )
  for (family in c(families, "baseline")) {
    cv <- probability_metrics(evidence$cv_predictions$DEFAULT, evidence$cv_predictions[[family]])
    heldout <- probability_metrics(runtime$predictions$DEFAULT, runtime$predictions[[family]])
    stopifnot(
      max(abs(cv - unlist(runtime$CV[runtime$CV$family == family, names(cv)]))) <
        1e-12, max(
        abs(
          heldout - unlist(runtime$metrics[runtime$metrics$family == family, names(heldout)])
        )
      ) <
        1e-12
    )
  }
  passed("Disjoint split, complete CV predictions and recomputed CV/holdout measures")
```

```
bootstrap <- evidence$bootstrap
stopifnot(
  bootstrap$B == 200L, bootstrap$successful == 200L, bootstrap$failures ==
    0L, ncol(evidence$bootstrap_indices) ==
    200L, abs(bootstrap$corrected_AUC - bootstrap$apparent_AUC + bootstrap$mean_optimism) <
    1e-12, all(
    vapply(
      bootstrap$replicates, function(replicate) {
        is.null(replicate$error) &&
          !length(replicate$diagnostics$warnings) &&
          replicate$diagnostics$boundary_probabilities == 0L && replicate$diagnostics$max_abs_coefficient <
          10
      }, logical(1)
    )
  )
)
passed("200 saved backward-procedure bootstrap records and diagnostics")

predictions <- vapply(
  runtime$presets, function(profile) {
    predict_raw(model, profile)
  }, numeric(1)
)
stopifnot(
  max(abs(predictions - runtime$preset_probabilities)) <
    1e-12
)
```

```
stopifnot(
  max(abs(predictions - c(0.095049785, 0.1554675612, 0.5304452853))) <
    1e-09
)
effects <- vapply(
  c("1", "2", "3+"),
  function(code) {
    profile <- runtime$presets[[3]]
    profile$PAY_0 <- code
    predict_raw(model, profile)
  }, numeric(1)
)
stopifnot(
  max(abs(effects - evidence$repayment_sensitivity$probability)) <
    1e-12
)
stopifnot(effects[2] > effects[3], effects[3] > effects[1])
passed("Three frozen presets and verified non-monotonic September effects")

profile <- runtime$presets[[1]]
for (variable in money_variables) {
  profile[[variable]] <- 0
}
for (variable in paste0("BILL_AMT", 1:6)) {
  profile[[variable]] <- -1000
}
check_probability(
  predict_raw(model, profile),
  1L
)
for (variable in status_variables) {
  for (code in c(
    as.character(-2:9),
    "3+"
  )) {
    changed <- profile
    changed[[variable]] <- code
    check_probability(
      predict_raw(model, changed),
      1L
    )
  }
}
for (variable in setdiff(candidate_variables, model$required)) {
  changed <- profile
  changed[[variable]] <- NA_real_
  stopifnot(
    abs(
      predict_raw(model, changed) -
        predict_raw(model, profile)
    ) <
      1e-12
  )
}
for (variable in model$required) {
  invalid <- profile
  invalid[[variable]] <- NA
  expect_error(predict_raw(model, invalid))
  invalid[[variable]] <- NULL
  expect_error(predict_raw(model, invalid))
}
invalid <- profile
invalid$AGE <- 17
expect_error(predict_raw(model, invalid))
```

```
  invalid <- profile
  invalid$LIMIT_BAL <- -1
  expect_error(predict_raw(model, invalid))
  invalid <- profile
  invalid$PAY_0 <- "unknown"
  expect_error(predict_raw(model, invalid))
  passed("Zero amounts, negative unused bills, all codes, unused and invalid inputs")

  probabilities <- runtime$predictions$lasso
  truth <- runtime$predictions$DEFAULT
  operating <- threshold_metrics(truth, probabilities, 0.5)
  stopifnot(
    all(
      operating[c("TP", "FN", "FP", "TN")] ==
        c(416, 912, 215, 4458)
    ),
```

```
    is.na(threshold_metrics(truth, probabilities, 1)["PPV"]),
    threshold_metrics(truth, probabilities, 0)["Flagged"] ==
      1, max(
      abs(
        probability_metrics(truth, probabilities) -
          unlist(
            runtime$metrics[runtime$metrics$family == "lasso", c("AUC", "LogLoss", "Brier")]
          )
      )
    ) <
      1e-12
  )
  passed("Threshold counts, extremes, undefined PPV and fixed probability metrics")
  stopifnot(identical(assets$coefficients, final_coefficient_table(model)))
  passed("Readable nonzero coefficient table including intercept and references")
  list(passed = TRUE, checks = checks, checked_at = format(Sys.time(), tz = "UTC"))
}

verify_analysis_outputs <- function(analysis_root) {
  for (package in c("MASS", "glmnet", "ranger", "pROC", "digest")) {
    if (!requireNamespace(package, quietly = TRUE)) {
      stop("Missing analysis package: ", package)
    }
  }
  results <- file.path(analysis_root, "results")
  cache <- readRDS(file.path(results, "analysis_cache_manifest.rds"))
  audit <- readRDS(file.path(results, "audit_bundle.rds"))
  lock <- readRDS(file.path(results, "model_selection.rds"))
  evaluation <- readRDS(file.path(results, "test_evaluation.rds"))
  model_paths <- file.path(results, "final", paste0(families, ".rds"))
  actual_models <- setNames(
    vapply(model_paths, sha256_file, character(1)),
    families
  )
  stopifnot(
    identical(actual_models, lock$models_hash),
    identical(
      evaluation$selection_lock_hash, sha256_file(file.path(results, "model_selection.rds"))
    ),
    identical(audit$manifest$code_hash, lock$manifest$code_hash),
    identical(audit$manifest$code_hash, evaluation$manifest$code_hash),
    identical(audit$manifest$config, config),
    identical(cache$data_hash, audit$manifest$data_hash)
  )

  # Check sealed historical bytes before accepting formatting-only migration.
  source_paths <- file.path(analysis_root, names(cache$sources))
  stopifnot(all(file.exists(source_paths)))
  actual_sources <- setNames(
    vapply(source_paths, sha256_file, character(1)),
    names(cache$sources)
  )
  stopifnot(identical(actual_sources, cache$sources))
  semantic_equivalence <- NULL
  if (!identical(audit$manifest$code_hash, code_fingerprint())) {
    historical <- new.env(parent = globalenv())
    for (path in sort(
      list.files(
        file.path(analysis_root, "R"),
        "[.]R$",
        full.names = TRUE
      )
    )) {
      sys.source(path, envir = historical, keep.source = FALSE)
    }
    semantic_equivalence <- vapply(
      analysis_function_names, function(name) {
        identical(
          function_signature(get(name, envir = historical)),
          function_signature(get(name, mode = "function"))
        )
      }, logical(1)
    )
    if (!all(semantic_equivalence)) {
      stop(
```

```
        "Changed statistical implementation: ", paste(
          names(semantic_equivalence)[!semantic_equivalence],
          collapse = ", "
        )
      )
```

```
    }
  }
  models <- setNames(
    lapply(model_paths, readRDS),
    families
  )
  for (model in models) {
    stopifnot(identical(model$manifest$code_hash, audit$manifest$code_hash))
  }
  dev <- readRDS(file.path(results, "development_data.rds"))
  test <- readRDS(file.path(results, "locked_test_data.rds"))
  allocation <- audit$allocation
  stopifnot(
    identical(dev$ID, allocation$development_ids),
    identical(test$ID, allocation$test_ids)
  )
  # Re-predict every holdout row with all five locked fits; no fitting occurs.
  for (family in families) {
    stopifnot(
      max(
        abs(
          predict_raw(models[[family]], test) -
            evaluation$predictions[[family]]
        )
      ) <
        1e-12
    )
  }
  outer_records <- lapply(
    1:5, function(fold) {
      record <- readRDS(file.path(results, "outer", sprintf("fold_%d.rds", fold)))
      stopifnot(
        !length(intersect(record$training_ids, record$assessment_ids)),
        setequal(record$training_ids, dev$ID[allocation$outer != fold]),
        identical(record$assessment_ids, dev$ID[allocation$outer == fold]),
        identical(record$inner_foldid, allocation$inner[[fold]])
      )
      for (family in families) {
        fit <- record$models[[family]]
        stopifnot(
          identical(fit$diagnostics$training_ids_hash, digest::digest(record$training_ids)),
          !length(fit$diagnostics$warnings)
        )
        if (family == "lasso") {
          stopifnot(identical(fit$settings$foldid, allocation$inner[[fold]]))
        }
        if (family == "forest") {
          stopifnot(identical(fit$settings$tuning$foldid, allocation$inner[[fold]]))
        }
      }
      list(
        training_ids = record$training_ids, assessment_ids = record$assessment_ids,
        inner_foldid = record$inner_foldid, diagnostics = lapply(record$models, function(model) model$diagnostics),
        retained = lapply(record$models, function(model) model$required),
        lasso_tuning = record$models$lasso$settings, forest_tuning = record$models$forest$settings$tuning,
        manifest = record$manifest
      )
    }
  )
  list(
    audit = audit, lock = lock, evaluation = evaluation, models = models, dev = dev,
    test = test, outer = outer_records, provenance = list(
      historical_training_manifest = audit$manifest, historical_cache_seal = cache,
      historical_model_file_sha256 = actual_models, current_analysis_fingerprint = code_fingerprint(),
      statistical_functions_unchanged = semantic_equivalence, migration = paste(
        "Historical manifests retained without rewriting.", "Original sealed source bytes checked; statistical function
        ↪  expressions",
        "compared independently after formatting. All five holdout predictions",
        "recomputed from saved fits; expensive training was not independently rerun."
      )
    )
  )
}

# 9. One explicit release operation ------------------------------------

package_release <- function(
  analysis_root, output_dir = .creditlens_root, deployment = NULL, screenshot = NULL,
  verification = list()
) {
  analysis_root <- normalizePath(analysis_root, winslash = "/", mustWork = TRUE)
```

```
output_dir <- normalizePath(output_dir, winslash = "/", mustWork = TRUE)
verified <- verify_analysis_outputs(analysis_root)
results <- file.path(analysis_root, "results")
read_result <- function(name) readRDS(file.path(results, name))
read_table <- function(name) {
  utils::read.csv(
    file.path(results, name),
    check.names = FALSE
  )
}
read_record <- function(name) {
  path <- file.path(results, name)
  if (file.exists(path)) {
    jsonlite::read_json(path, simplifyVector = TRUE)
  } else {
    NULL
  }
}
read_bytes <- function(path) readBin(path, "raw", n = file.info(path)$size)
model <- verified$models[[verified$lock$chosen]]
original_id <- model_identity(model)
model$manifest <- NULL
model$diagnostics <- NULL
environment(model$prep$formula) <- baseenv()
model$engine$call <- NULL
model$settings <- model$settings[c("lambda", "lambda_min", "coefficients")]
stopifnot(
  identical(
    model_identity(model),
    original_id
  )
)

dev <- verified$dev
development_probabilities <- predict_raw(model, dev)
targets <- quantile(development_probabilities, c(0.1, 0.5, 0.9))
rows <- vapply(
  targets, function(target) {
    which.min(abs(development_probabilities - target))
  }, integer(1)
)
presets <- setNames(
  lapply(
    rows, function(row) {
      profile <- dev[row, candidate_variables, drop = FALSE]
      rownames(profile) <- NULL
      for (variable in status_variables) {
        profile[[variable]] <- group_status(profile[[variable]])
      }
      profile
    }
  ),
  c(
    "Lower predicted probability", "Middle predicted probability", "Higher predicted probability"
  )
)

source_hash <- sha256_file(file.path(output_dir, "CreditLens.R"))
report_hash <- sha256_file(file.path(output_dir, "individual_assignment.Rmd"))
identity <- list(
  model_id = original_id, release_id = paste0(
    "CL-", substr(
      digest::digest(
        list(
          model = original_id, source = source_hash, report = report_hash,
          lock = sha256_file(file.path(results, "model_selection.rds"))
        ),
        algo = "sha256"
      ),
      1, 12
    )
  ),
  selected_family = verified$lock$chosen, training_sample = "Complete development only; no test refit",
  training_lock_at = verified$lock$locked_at
)
if (is.null(deployment)) {
  deployment <- list(status = "unverified", url = NULL)
}
if (!is.null(deployment$identity)) {
  stopifnot(identical(deployment$identity, identity))
```

```
}
deployment$identity <- identity
predictions <- verified$evaluation$predictions
predictions$ID <- NULL
rownames(predictions) <- NULL
dictionary <- make_dictionary()
audit <- verified$audit$audit
audit$file <- basename(audit$file)
audit$status_ambiguity <- NULL
audit$repayment_coding <- repayment_metadata(model)
runtime <- list(
```

```
  model = model, presets = presets, preset_probabilities = vapply(
    presets, function(profile) predict_raw(model, profile),
    numeric(1)
  ),
  predictions = predictions, metrics = verified$evaluation$metrics, thresholds = verified$evaluation$thresholds,
  calibration = verified$evaluation$calibration, calibration_numbers = verified$evaluation$calibration_numbers,
  CV = read_table("cv_metrics.csv"),
  importance = read_table("permutation_importance.csv"),
  stability = read_table("bootstrap_stability.csv"),
  dictionary = dictionary, audit = audit[c("rows", "events", "non_events", "missing", "unique_ids")],
  development_n = nrow(dev),
  test_n = nrow(predictions),
  development_events = sum(dev$DEFAULT == "Yes"),
  data_hash = audit$sha256, selection_timestamp = verified$lock$locked_at,
  chosen = verified$lock$chosen, label = unname(model_labels[verified$lock$chosen]),
  identity = identity
)
coefficients <- final_coefficient_table(model)
runtime$coefficients <- coefficients
runtime$repayment <- repayment_metadata(model)
runtime$selection <- verified$lock$metrics

# Package supporting records directly, without a supplementary directory.
evidence <- list(
  dataset_provenance = audit, provenance_comparison = read_record("provenance_comparison.json"),
  allocations = verified$audit$allocation, selection_lock = verified$lock,
  selection_lock_sha256 = sha256_file(file.path(results, "model_selection.rds")),
  cv_predictions = read_result("oof_predictions.rds"),
  cv_fold_metrics = read_table("cv_fold_metrics.csv"),
  cv_calibration = read_table("cv_calibration.csv"),
  holdout = verified$evaluation, outer_fold_records = verified$outer, diagnostics = lapply(verified$models, function(model)
  ↪  model$diagnostics),
  bootstrap = read_result("bootstrap_summary.rds"),
  bootstrap_indices = read_result("bootstrap_indices.rds"),
  permutation_importance = read_table("permutation_importance.csv"),
  permutation_detail = read_table("permutation_detail.csv"),
  full_coefficients = read_table("full_coefficients.csv"),
  full_joint_tests = read_table("full_joint_tests.csv"),
  full_IQR_effects = read_table("full_IQR_effects.csv"),
  development_numeric = read_table("development_numeric.csv"),
  development_categories = read_table("development_categories.csv"),
  development_status_rates = read_table("development_status_rates.csv"),
  specifications = read_table("development_specifications.csv"),
  tuning = list(
    lasso = verified$models$lasso$settings, lasso_sensitivity = read_table("lasso_development_sensitivity.csv"),
    forest = verified$models$forest$settings$tuning
  ),
  pooling_correction = read_record("pooling_repair.json"),
  pooling_encoding_comparison = if (file.exists(file.path(results, "pooling_cache_compatibility.csv"))) {
    read_table("pooling_cache_compatibility.csv")
  } else {
    NULL
  }, training_provenance = verified$provenance, dictionary = dictionary
)
evidence$repayment_sensitivity <- data.frame(
  September = c("1", "2", "3+"),
  probability = vapply(
    c("1", "2", "3+"),
    function(code) {
      profile <- presets[[3]]
      profile$PAY_0 <- code
      predict_raw(model, profile)
    }, numeric(1)
  )
)
objects <- list(
  a = list(audit = audit),
  lock = verified$lock, e = verified$evaluation, boot = evidence$bootstrap,
  model = model, cv = runtime$CV, numeric_summary = evidence$development_numeric,
  status_summary = evidence$development_status_rates, spec = evidence$specifications,
```

```
  importance = runtime$importance, stability = runtime$stability, deployment = deployment,
  dev_n = runtime$development_n, test_n = runtime$test_n, dev_events = runtime$development_events,
  test_events = sum(predictions$DEFAULT == "Yes"),
  families = families, model_labels = model_labels, repayment_sensitivity = evidence$repayment_sensitivity,
  coefficients = coefficients, identity = identity
)
# Generate figures from this release's actual saved predictions and summaries.
figure_directory <- tempfile("creditlens-figures-")
dir.create(figure_directory)
on.exit(
  unlink(figure_directory, recursive = TRUE),
  add = TRUE
)
make_release_figures(runtime, evidence, figure_directory, dev)
figure_names <- c("development_limit.png", "development_status.png")
app_figures <- setNames(
  lapply(
    figure_names, function(name) {
      read_bytes(file.path(figure_directory, name))
    }
  ),
```

```
    figure_names
  )
  report_figures <- list(
    roc_calibration = read_bytes(file.path(figure_directory, "roc_calibration.png"))
  )
  if (!is.null(screenshot)) {
    report_figures$app_predict <- read_bytes(screenshot)
  }
  assets <- list(
    schema = 2L, identity = identity, source_sha256 = source_hash, report_sha256 = report_hash,
    app = list(runtime = runtime, figures = app_figures),
    report = list(objects = objects, figures = report_figures, identity = identity),
    coefficients = coefficients, evidence = evidence, dependencies = tested_dependencies(),
    deployment = deployment, verification = verification
  )
  assets$integrity <- asset_checksum(assets)
  assets$verification$frozen <- verify_release(assets = assets)
  assets$integrity <- asset_checksum(assets)
  save_atomic(assets, file.path(output_dir, "creditlens_assets.rds"))
  message("Packaged ", identity$release_id, "; model ", substr(original_id, 1, 12))
  invisible(assets)
}

make_release_figures <- function(runtime, evidence, directory, development_data) {
  limit_plot <- ggplot2::ggplot(development_data, ggplot2::aes(DEFAULT, LIMIT_BAL / 1000, fill = DEFAULT)) +
    ggplot2::geom_boxplot(outlier.alpha = 0.1) +
    ggplot2::theme_minimal() +
    ggplot2::scale_fill_manual(values = c("#2C7FB8", "#D95F0E")) +
    ggplot2::labs(
      x = "Recorded next-month default", y = "Credit limit (thousand NT dollars)",
      title = "Development sample", fill = NULL
    )
  ggplot2::ggsave(
    file.path(directory, "development_limit.png"),
    limit_plot,
    width = 6, height = 3.5, dpi = 180
  )
  rates <- evidence$development_status_rates
  rates$status <- factor(rates$status, levels = status_levels)
  status_plot <- ggplot2::ggplot(rates, ggplot2::aes(status, rate)) +
    ggplot2::geom_col(fill = "#2C7FB8") +
    ggplot2::geom_errorbar(
      ggplot2::aes(ymin = lower, ymax = upper),
      width = 0.15
    ) +
    ggplot2::geom_text(
      ggplot2::aes(label = paste0("n=", n)),
      vjust = -0.8, size = 3
    ) +
    ggplot2::theme_minimal() +
    ggplot2::labs(
      x = "September category (-2 no consumption; -1 paid in full; 0 revolving)",
      y = "Observed default proportion", title = "Development sample; exact binomial 95% intervals"
    )
  ggplot2::ggsave(
    file.path(directory, "development_status.png"),
    status_plot,
    width = 7, height = 3.5, dpi = 180
  )
```

```
  families_plot <- c(runtime$chosen, "forest")
  colours <- c("#2C7FB8", "#D95F0E")
  roc_data <- do.call(
    rbind, lapply(
      families_plot, function(family) {
        roc <- roc_object(runtime$predictions$DEFAULT, runtime$predictions[[family]])
        data.frame(
          family = unname(model_labels[family]),
          FPR = 1 - roc$specificities, Sensitivity = roc$sensitivities
        )
      }
    )
  )
  bins <- do.call(
    rbind, lapply(
      families_plot, function(family) {
        data.frame(
          family = unname(model_labels[family]),
          runtime$calibration[[family]]
        )
      }
    )
  )
  roc_data$family <- factor(roc_data$family, levels = unname(model_labels[families_plot]))
  bins$family <- factor(bins$family, levels = levels(roc_data$family))
  roc_plot <- ggplot2::ggplot(roc_data, ggplot2::aes(FPR, Sensitivity, colour = family)) +
    ggplot2::geom_line(linewidth = 0.8) +
    ggplot2::geom_abline(linetype = "dashed", colour = "grey65") +
    ggplot2::scale_colour_manual(values = colours) +
    ggplot2::coord_equal(
      xlim = c(0, 1),
```

```
      ylim = c(0, 1)
    ) +
    ggplot2::theme_minimal(base_size = 11) +
    ggplot2::theme(legend.position = "bottom") +
    ggplot2::labs(
      x = "1 - Specificity", y = "Sensitivity", colour = NULL, title = "Final test: ROC"
    )
  calibration_plot <- ggplot2::ggplot(bins, ggplot2::aes(predicted, observed, colour = family)) +
    ggplot2::geom_abline(linetype = "dashed", colour = "grey65") +
    ggplot2::geom_errorbar(
      ggplot2::aes(ymin = lower, ymax = upper),
      width = 0.012
    ) +
    ggplot2::geom_point(size = 1.8) +
    ggplot2::scale_colour_manual(values = colours) +
    ggplot2::coord_equal(
      xlim = c(0, 1),
      ylim = c(0, 1)
    ) +
    ggplot2::theme_minimal(base_size = 11) +
    ggplot2::theme(legend.position = "bottom") +
    ggplot2::labs(
      x = "Mean predicted probability", y = "Observed default proportion",
      colour = NULL, title = "Final test: calibration"
    )
  grDevices::png(
    file.path(directory, "roc_calibration.png"),
    width = 1500, height = 600, res = 180
  )
  grid::grid.newpage()
  grid::pushViewport(grid::viewport(layout = grid::grid.layout(1, 2)))
  print(roc_plot, vp = grid::viewport(layout.pos.row = 1, layout.pos.col = 1))
  print(calibration_plot, vp = grid::viewport(layout.pos.row = 1, layout.pos.col = 2))
  grDevices::dev.off()
}

# 10. Frozen startup, rendering and deployment ------------------------------

render_creditlens_report <- function(root = .creditlens_root) {
  load_creditlens_assets(root)
  if (!rmarkdown::pandoc_available()) {
    stop(
      "Pandoc was not discovered. Render in RStudio, add Pandoc to PATH, or set RSTUDIO_PANDOC."
    )
  }
  rmarkdown::render(
    file.path(root, "individual_assignment.Rmd"),
    output_format = "pdf_document", envir = new.env(parent = globalenv()),
```

```
    clean = TRUE
  )
}

deploy_creditlens <- function(root = .creditlens_root, account = NULL) {
  assets <- load_creditlens_assets(root)
  verify_release(assets = assets)
  accounts <- rsconnect::accounts()
  accounts <- accounts[accounts$server == "shinyapps.io", , drop = FALSE]
  if (!is.null(account)) {
    accounts <- accounts[accounts$name == account, , drop = FALSE]
  } else if (!is.null(assets$deployment$account)) {
    accounts <- accounts[accounts$name == assets$deployment$account, , drop = FALSE]
  }
  if (nrow(accounts) !=
    1L) {
    stop("Specify one configured shinyapps.io account with account = ...")
  }
  bundle <- tempfile("creditlens-deployment-")
  dir.create(bundle)
  on.exit(
    unlink(bundle, recursive = TRUE),
    add = TRUE
  )
  stopifnot(
    file.copy(
      file.path(root, c("CreditLens.R", "creditlens_assets.rds")),
      bundle
    )
  )
  writeLines(
    c(
      "source(\"CreditLens.R\", local = TRUE)", "create_creditlens_app(root = getwd())"
    ),
    file.path(bundle, "app.R")
  )
  # Only these three files are uploaded. No development output is read at startup.
  rsconnect::deployApp(
    appDir = bundle, appFiles = c("app.R", "CreditLens.R", "creditlens_assets.rds"),
    appName = "creditlens", account = accounts$name[1], server = accounts$server[1],
    launch.browser = FALSE, logLevel = "normal", forceUpdate = TRUE
  )
  applications <- rsconnect::applications(account = accounts$name[1], server = accounts$server[1])
```

```
  application <- applications[applications$name == "creditlens", , drop = FALSE]
  stopifnot(
    nrow(application) ==
      1L, nzchar(application$url[1])
  )
  list(
    status = "deployed_unverified", url = application$url[1], account = accounts$name[1],
    app_id = application$id[1], identity = assets$identity, deployed_at = format(Sys.time(), tz = "UTC"),
    bundle_files = c("app.R", "CreditLens.R", "creditlens_assets.rds")
  )
}

creditlens <- function(
  action = "app", root = .creditlens_root, ..., host = "127.0.0.1", port = getOption("shiny.port", 3838),
  launch.browser = interactive()
) {
  switch(action,
    app = shiny::runApp(
      create_creditlens_app(root),
      host = host, port = port, launch.browser = launch.browser
    ),
    load = load_creditlens_assets(root),
    verify = verify_release(root),
    report = render_creditlens_report(root),
    deploy = deploy_creditlens(root, ...),
    inspect = inspect_creditlens(root = root, ...),
    stop("Choose app, load, verify, report, deploy or inspect.")
  )
}


# 11. Shiny application ----------------------------------------------------

input_label <- function(variable) {
  months <- c("September", "August", "July", "June", "May", "April")
  if (variable == "AGE") {
    return("Age (years)")
  }
  if (variable == "LIMIT_BAL") {
    return("Credit limit (NT$)")
  }
  if (variable %in% status_variables) {
    return(
      paste(
        months[match(variable, status_variables)],
        "repayment status"
      )
    )
  }
  paste(
    months[match(variable, paste0("PAY_AMT", 1:6))],
    "previous payment (NT$)"
  )
}

scenario_changes <- function(profile_a, profile_b, model) {
  labels <- c(
    `-2` = "No consumption", `-1` = "Paid in full", `0` = "Revolving credit",
    `1` = "1 month delayed", `2` = "2 months delayed", `3+` = "3+ months delayed"
  )
  rows <- lapply(
    model$required, function(variable) {
      a <- profile_a[[variable]]
      b <- profile_b[[variable]]
      if (variable %in% status_variables) {
        a <- unname(model$prep$status_mapping[[variable]][group_status(a)])
        b <- unname(model$prep$status_mapping[[variable]][group_status(b)])
        if (identical(a, b)) {
          return(NULL)
        }
        choices <- labels
        if (model$prep$status_mapping[[variable]]["1"] == "2") {
          choices["2"] <- "1-2 months delayed (pooled)"
        }
        a <- unname(choices[a])
        b <- unname(choices[b])
      } else {
        if (isTRUE(all.equal(a, b))) {
          return(NULL)
        }
        a <- format(a, big.mark = ",", scientific = FALSE, trim = TRUE)
        b <- format(b, big.mark = ",", scientific = FALSE, trim = TRUE)
      }
      data.frame(
        Input = input_label(variable),
        A = a, B = b
      )
    }
  )
  result <- do.call(rbind, rows)
  if (is.null(result)) {
```

```
    result <- data.frame(Input = character(), A = character(), B = character())
  }
  result
}


create_creditlens_app <- function(root = .creditlens_root, assets = NULL) {
  library(shiny)
  library(bslib)

  if (is.null(assets)) {
    assets <- load_creditlens_assets(root)
  }

  rt <- assets$app$runtime
  final_model <- rt$model

  default_preset <- names(rt$presets)[2]
  default_profile <- rt$presets[[default_preset]]
  months <- c("September", "August", "July", "June", "May", "April")
  pct <- function(x) {
    if (length(x) ==
      1L && is.finite(x)) {
      sprintf("%.1f%%", 100 * x)
    } else {
      "Not estimable"
```

```
    }
  }
  status_choices <- function(v) {
    labels <- c(
      `-1` = "-1: Paid in full", `-2` = "-2: No consumption", `0` = "0: Revolving credit",
      `1` = "1 month delayed", `2` = "2 months delayed", `3+` = "3+ months delayed"
    )
    lev <- final_model$prep$factor_levels[[v]]
    if (final_model$prep$status_mapping[[v]]["1"] == "2") {
      labels["2"] <- "1-2 months delayed (pooled)"
    }
    setNames(lev, unname(labels[lev]))
  }
  input_field <- function(v, label) {
    used <- v %in% final_model$required
    label <- tags$span(
      label, if (!used) {
        tags$small(class = "unused-label", " (unused)")
      }
    )
    x <- if (v %in% status_variables) {
      value <- unname(final_model$prep$status_mapping[[v]][group_status(default_profile[[v]])])
      selectInput(
        v, label,
        choices = status_choices(v),
        selected = value, selectize = FALSE
      )
    } else {
      numericInput(
        v, label,
        value = default_profile[[v]], min = if (v == "AGE") {
          18
        } else if (v == "LIMIT_BAL" || startsWith(v, "PAY_AMT")) {
          0
        } else {
          NA
        }, step = if (v == "AGE") {
          1
        } else {
          1000
        }
      )
    }
    if (!used) {
      x <- htmltools::tagQuery(x)$find(
        if (v %in% status_variables) {
          "select"
        } else {
          "input"
        }
      )$addAttrs(disabled = "disabled")$allTags()
    }
    x
  }
  month_card <- function(j) {
    card(
      class = "month-card", fill = FALSE, card_header(
        paste0(
          months[j], " 2005", if (j == 1) {
            " | most recent"
          } else {
            ""
          }
        )
      ),
```

```
      input_field(status_variables[j], "Repayment status"),
      input_field(
        paste0("PAY_AMT", j),
        "Previous payment (NT$)"
      )
    )
  }
  css <- "
body {background:#f3f6fa;color:#18334c;}
.navbar {border-bottom:3px solid #2c7fb8;}
.navbar-brand {font-weight:750;letter-spacing:-.5px;font-size:1.55rem;}
.hero {padding:20px 0 12px;max-width:900px;}
.hero h2 {font-weight:700;letter-spacing:-.6px;}
.eyebrow {color:#2c7fb8;font-weight:700;text-transform:uppercase;font-size:.78rem;letter-spacing:1.2px;}
.card {border:1px solid #dde5ef;border-radius:12px;box-shadow:0 3px 12px #16334c08;}
```

```
.card-header {font-weight:650;background:#eaf1f8;}
.month-grid {display:grid;grid-template-columns:repeat(3,minmax(0,1fr));gap:12px;}
.month-card .card-body {padding:12px;}
.month-card .form-group {margin-bottom:8px;}
.month-card .form-control {font-size:.88rem;}
.unused-label {font-weight:400;color:#788898;}
input:disabled {background:#eef1f4;color:#788898;}
.probability {font-size:3.3rem;font-weight:750;line-height:1.15;color:#2c7fb8;}
.muted {color:#627487;font-size:.9rem;}
.note {background:#eaf1f8;border-left:3px solid #2c7fb8;padding:12px;border-radius:4px;}
.support-note {background:#fff4e6;border-left:3px solid #d95f0e;padding:10px;}
.error-note {color:#9d2732;background:#ffedf0;padding:12px;border-radius:8px;}
.scenario-a {color:#d95f0e;font-weight:650;}
.scenario-b {color:#2c7fb8;font-weight:650;}
.actions {display:flex;gap:8px;flex-wrap:wrap;margin:8px 0 14px;}
.table {font-size:.88rem;}
.dictionary {max-height:460px;overflow:auto;}
.table-wrap {max-width:100%;overflow-x:auto;}
.data-figures {display:grid;grid-template-columns:1fr 1fr;gap:12px;}
.data-figures img {max-width:100%;}
@media(max-width:900px) {.month-grid {grid-template-columns:repeat(2,minmax(0,1fr));}}
@media(max-width:550px) {.month-grid,.data-figures {grid-template-columns:1fr;}.probability{font-size:2.7rem;}}
"
  selection_explanation <- paste(
    "LASSO was selected before opening the holdout: CV log loss 0.4355 and AUC 0.7722",
    "qualified within 1% of the forest's minimum loss (0.4324) and within 0.01 AUC.",
    "It required 14 inputs, versus 16 for backward AIC and 20 for full logistic and the forest.",
    "Fewer entries and an inspectable additive log-odds structure motivated this trade-off.",
    "The forest subsequently performed better on the holdout; the original choice is retained."
  )
  ui <- page_navbar(
    title = "CreditLens", id = "page", fillable = FALSE, theme = bs_theme(
      version = 5, primary = "#2C7FB8", secondary = "#D95F0E", base_font = "system-ui"
    ),
    footer = div(
      class = "muted", id = "release_identity", paste(
        "Release", rt$identity$release_id, "| Model", substr(rt$identity$model_id, 1, 12)
      )
    ),
    header = tagList(
      tags$style(HTML(css)),
      tags$script(
        HTML(
          paste0(
            "$(document).on('click', '#load_preset, #reset', function(){ ",
            "document.getElementById('predict').disabled=true; }); ", "$(document).on('shiny:connected', function(){ ",
            "Shiny.addCustomMessageHandler('creditlens_profile_loading', ",
            "function(loading){ document.getElementById('predict').disabled=loading; }); ",
            "});"
          )
        )
      )
    ),
    nav_panel(
      "Data & problem",
      value = "data", div(
        class = "hero", div(class = "eyebrow", "Historical credit-card payment default"),
        h2("Six months of history. One next-month prediction."),
        p(
          paste0(
            "Explore how payment histories relate to the recorded default-payment outcome, ",
            "and how validation changes our confidence in a model."
          )
        )
      ),
      layout_columns(
        col_widths = c(8, 4),
        card(
          fill = FALSE, card_header("The prediction problem"),
          p(
            paste0(
              "Using information at the end of September 2005, how accurately can we predict ",
              "the recorded default-payment outcome in the following month?"
            )
          ),
          p(
```

```
          paste0(
            "Each row represents one historical Taiwanese credit-card customer. The source ",
            "reports April-September 2005 histories. Monetary amounts are New Taiwan ",
            "dollars (NT$); age is in years."
```

```
          )
        ),
        p(
          paste0(
            "Outcome: 0 = no recorded default payment; 1 = recorded default payment. Its ",
            "exact operational definition is incompletely described by the supplier."
          )
        ),
        p(
          tags$a(
            href = "https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients",
            target = "_blank", "UCI dataset and dictionary"
          ),
          paste0(
            " | Yeh (2009), DOI 10.24432/C55S3H | CC BY 4.0 | local Kaggle CSV verified ",
            "against the original UCI workbook."
          )
        )
      ),
      card(
        fill = FALSE, card_header("Audited sample"),
        h3(format(rt$audit$rows, big.mark = ",")),
        p("unique customers | no missing values"),
        p(
          paste(
            format(rt$audit$events, big.mark = ","),
            "recorded defaults;", pct(rt$audit$events / rt$audit$rows),
            "of the full sample."
          )
        ),
        p(
          paste(
            format(rt$development_n, big.mark = ","),
            "development /", format(rt$test_n, big.mark = ","),
            "final test."
          )
        ),
        p(tags$strong(paste("Final model:", rt$label))),
        p(
          paste(
            length(final_model$required),
            "original inputs; selected using nested development CV before test evaluation."
          )
        ),
        p(selection_explanation)
      )
    ),
    card(
      fill = FALSE, card_header("Predictors and coding"),
      p(
        paste0(
          "Twenty original candidates: credit limit, age, six repayment statuses, six ",
          "bills and six previous payments. ID is excluded. Sex, education and marital ",
          "status are omitted to focus this primary analysis on history and financial ",
          "information; this does not establish fairness."
        )
      ),
      p(
        paste0(
          "Repayment status is categorical. Codes 3-9 form 3+. Code -2 means no ",
          "consumption, -1 paid in full (abbreviated UCI: paid duly), and 0 use of ",
          "revolving credit. The expanded published coding supplements the abbreviated ",
          "UCI description; the three codes remain distinct. Within each training fit, a ",
          "one-month-delay category with fewer than 50 observations is pooled into the ",
          "two-month category. The fitted mapping is applied to new profiles."
        )
      ),
      p(
        tags$a(
          href = "https://arxiv.org/pdf/2209.10070", target = "_blank",
          "Chen and Ye (2022), section 4.1.1"
        ),
        " and ", tags$a(
          href = "https://h2oai.github.io/tutorials/machine-learning-interpretability/",
          target = "_blank", "H2O.ai tutorial"
        ),
        " provide the expanded labels."
      ),
      p(
        paste0(
```

```
          "Fitted mapping: September retains code 1; August-April pool raw codes 1 and ",
          "2. Codes -2, -1 and 0 stay separate. No numerical delay scale is imposed."
        )
      ),
```

```
      tags$details(
        tags$summary("Inspect final LASSO coefficients"),
        p(
          paste0(
            "Additive log-odds coefficients; -1 paid in full is the status reference. Zero ",
            "terms are omitted. Penalized coefficients do not have ordinary unpenalized ",
            "p-values."
          )
        ),
        div(class = "table-wrap", tableOutput("coefficients"))
      ),
      p(
        paste0(
          "Monetary predictors use sign(x) * log(1 + |x|/1000), preserving zero and ",
          "negative bills. Age stays numerical. Complete data required no imputation."
        )
      ),
      div(class = "dictionary", tableOutput("dictionary"))
    ),
    tags$details(
      tags$summary("Development-sample descriptions"),
      div(
        class = "data-figures", tags$img(
          src = paste0(
            "", base64enc::base64encode(assets$app$figures[["development_limit.png"]])
          ),
          alt = "Development credit-limit distributions by recorded outcome"
        ),
        tags$img(
          src = paste0(
            "", base64enc::base64encode(assets$app$figures[["development_status.png"]])
          ),
          alt = "Development default proportions by September repayment status with binomial intervals"
        )
      )
    ),
    div(
      class = "note", p(
        paste0(
          "Educational exploration of a historical sample. Internal validation does not ",
          "establish current performance in another bank or country. These predictions ",
          "are associations and are not a validated credit-approval policy."
        )
      )
    )
  ),
  nav_panel(
    "Predict & compare",
    value = "predict", div(
      class = "hero", div(class = "eyebrow", paste(rt$label, "| fixed development fit")),
      h2("Explore a profile, then compare two scenarios."),
      p(selection_explanation),
      p(
        paste0(
          "The 14 active inputs are age, credit limit, six monthly repayment statuses ",
          "and six previous payments. Enter raw NT dollars. Six bill candidates were ",
          "excluded by this fitted LASSO."
        )
      )
    ),
    layout_columns(
      col_widths = c(8, 4),
      div(
        card(
          fill = FALSE, card_header("A real development profile to start from"),
          layout_columns(
            col_widths = c(8, 4),
            selectInput(
              "preset", "Profile",
              choices = names(rt$presets),
              selected = default_preset, selectize = FALSE
            ),
            div(
              class = "actions", actionButton("load_preset", "Load profile"),
              actionButton("reset", "Reset")
            )
```

```
          ),
          layout_columns(
            col_widths = c(6, 6),
            input_field("AGE", "Age (years)"),
            input_field("LIMIT_BAL", "Credit limit (NT$)")
          )
        ),
        div(
          class = "note", id = "active_inputs", tags$strong("14 active inputs"),
          p("Age and credit limit; September-April repayment status and previous payment.")
        ),
        div(class = "month-grid", lapply(1:6, month_card)),
        tags$details(
          id = "excluded_candidates", tags$summary("Six excluded bill candidates (not used for prediction)"),
          p(
```

```
            paste0(
              "The fitted LASSO set all six bill coefficients to zero. These fields remain ",
              "disabled; valid negative bills and zero amounts were retained during analysis."
            )
          ),
          lapply(
            1:6, function(j) {
              input_field(
                paste0("BILL_AMT", j),
                paste(months[j], "bill (NT$)")
              )
            }
          )
        )
      ),
      card(
        fill = FALSE, card_header("Recorded default payment in the following month"),
        div(
          class = "actions", actionButton("predict", "Predict", class = "btn-primary"),
          actionButton("save_a", "Save current as A")
        ),
        uiOutput("prediction"),
        uiOutput("support"),
        uiOutput("comparison"),
        div(class = "table-wrap", tableOutput("changed_inputs")),
        plotOutput("scenario_plot", height = "200px"),
        p(
          class = "muted", paste0(
            "Changing an input describes a different model profile. The difference is not ",
            "an estimated causal effect of an intervention. No individual probability ",
            "interval is claimed. Delay-category effects need not increase: the nominal ",
            "fit is unconstrained, and severe delays are sparse and correlated with other ",
            "histories."
          )
        )
      )
    )
  ),
  nav_panel(
    "Explore performance",
    value = "performance", div(
      class = "hero", div(class = "eyebrow", "Locked final test sample"),
      h2("Ranking, calibration and threshold trade-offs."),
      p(
        paste(
          format(rt$test_n, big.mark = ","),
          "customers;", format(
            sum(rt$predictions$DEFAULT == "Yes"),
            big.mark = ","
          ),
          "recorded defaults. All five models were fixed using development data."
        )
      )
    )
  ),
  layout_columns(
    col_widths = c(4, 8),
    card(
      fill = FALSE, card_header("Explore a fixed model"),
      selectInput(
        "perf_model", "Comparison model",
        choices = setNames(families, model_labels[families]),
        selected = rt$chosen, selectize = FALSE
      ),
      uiOutput("performance_summary"),
      sliderInput(
```

```
        "threshold", "Flag default when probability >= threshold",
        min = 0,
        max = 1, value = 0.5, step = 0.01
      ),
      uiOutput("threshold_summary"),
      tableOutput("confusion"),
      tags$details(
        tags$summary("What do these measures mean?"),
        p(
          paste0(
            "Sensitivity: the fraction of observed defaults flagged. Specificity: the ",
            "fraction of non-defaults correctly left unflagged. PPV: the fraction of ",
            "flagged customers who actually defaulted; undefined if nobody is flagged."
          )
        ),
        p(
          paste0(
            "Log loss penalizes incorrect probabilities, especially confident errors. ",
            "Brier score is the mean squared probability error. Lower is better for both. ",
            "AUC describes ranking and stays fixed when the threshold changes."
          )
        )
      ),
      p(
        class = "muted", paste0(
          "Threshold exploration changes classifications, not probabilities, AUC or the ",
```

```
            "final model on the prediction page. No optimal threshold is selected from ",
            "these test records."
          )
        )
      ),
      div(
        layout_columns(
          col_widths = c(6, 6),
          card(
            fill = FALSE, card_header("Test ROC"),
            plotOutput("roc_plot", height = "300px")
          ),
          card(
            fill = FALSE, card_header("Test calibration"),
            plotOutput("cal_plot", height = "300px")
          )
        ),
        card(
          fill = FALSE, card_header("All fixed models and the prevalence baseline"),
          div(class = "table-wrap", tableOutput("all_metrics")),
          p(
            class = "muted", paste0(
              "AUC intervals use DeLong and condition on the trained models. Calibration ",
              "points use equal-count groups and exact binomial intervals. Log loss is ",
              "clipped only for numerical logarithms at 1e-15."
            )
          )
        )
      )
    )
  )
)
server <- function(input, output, session) {
  current <- reactiveVal(NULL)
  scenario_a <- reactiveVal(NULL)
  validation_error <- reactiveVal(NULL)
  pending_profile <- reactiveVal(NULL)
  profile <- reactive({
    values <- setNames(
      lapply(
        candidate_variables, function(v) {
          x <- input[[v]]
          if (is.null(x) ||
            length(x) !=
              1L) {
            if (v %in% status_variables) {
              NA_character_
            } else {
              NA_real_
            }
          } else {
            x
          }
        }
```

```
      ),
      candidate_variables
    )
    as.data.frame(values, stringsAsFactors = FALSE)
  })
  active_profile <- reactive(profile()[final_model$required])
  same_profile <- function(p, target) {
    all(
      vapply(
        final_model$required, function(v) {
          x <- p[[v]]
          y <- target[[v]]
          if (v %in% status_variables) {
            !is.na(x) &&
              as.character(x) ==
                as.character(y)
          } else {
            is.finite(x) &&
              x == y
          }
        }, logical(1)
      )
    )
  }
  # Wait for the loaded inputs to reach the server before enabling prediction.
  observeEvent(
    active_profile(),
    {
      current(NULL)
      validation_error(NULL)
      if (!is.null(pending_profile()) &&
        same_profile(active_profile(), pending_profile())) {
        pending_profile(NULL)
        session$sendCustomMessage("creditlens_profile_loading", FALSE)
      }
    },
    ignoreInit = TRUE
```

```
)
load_profile <- function(p) {
  for (v in status_variables) p[[v]] <- unname(final_model$prep$status_mapping[[v]][group_status(p[[v]])])
  pending_profile(p)
  session$sendCustomMessage("creditlens_profile_loading", TRUE)
  for (v in candidate_variables) {
    if (v %in% status_variables) {
      updateSelectInput(
        session, v,
        selected = unname(final_model$prep$status_mapping[[v]][group_status(p[[v]])])
      )
    } else {
      updateNumericInput(session, v, value = p[[v]])
    }
  }
  current(NULL)
  validation_error(NULL)
  if (same_profile(active_profile(), p)) {
    pending_profile(NULL)
    session$sendCustomMessage("creditlens_profile_loading", FALSE)
  }
}
observeEvent(
  input$load_preset,
  {
    load_profile(rt$presets[[input$preset]])
  }
)
observeEvent(
  input$reset,
  {
    updateSelectInput(session, "preset", selected = default_preset)
    scenario_a(NULL)
    load_profile(default_profile)
  }
)
observeEvent(
  input$predict,
  {
    if (!is.null(pending_profile())) {
      return()
    }
    validation_error(NULL)
```

```
    tryCatch(
      {
        p <- profile()
        validate_raw(p, final_model$required)
        current(list(profile = p, probability = predict_raw(final_model, p)))
      },
      error = function(e) {
        current(NULL)
        validation_error(conditionMessage(e))
      }
    )
  }
)
observeEvent(
  input$save_a,
  {
    if (is.null(current())) {
      showNotification("Predict the current profile before saving scenario A.", type = "message")
    } else {
      scenario_a(current())
    }
  }
)
output$prediction <- renderUI({
  if (!is.null(pending_profile())) {
    return(p(class = "muted", "Loading the profile..."))
  }
  if (!is.null(validation_error())) {
    return(div(class = "error-note", validation_error()))
  }
  if (is.null(current())) {
    return(p(class = "muted", "Press Predict for the current profile."))
  }
  tagList(
    div(class = "probability", id = "current_probability", pct(current()$probability)),
    p("Predicted probability of the recorded next-month default-payment outcome.")
  )
})
output$support <- renderUI({
  req(current())
  p <- current()$profile
  outside <- vapply(
    intersect(final_model$required, names(final_model$prep$ranges)),
    function(v) {
      bounds <- final_model$prep$ranges[[v]]
      p[[v]] < bounds[1] || p[[v]] > bounds[2]
    }, logical(1)
  )
```

```
    if (any(outside)) {
      div(
        class = "support-note", paste(
          "Outside development support:", paste(
            names(outside)[outside],
            collapse = ", "
          ),
          ". This valid-domain profile requires extrapolation; its probability may be less reliable."
        )
      )
    }
  })
  output$comparison <- renderUI({
    if (is.null(scenario_a())) {
      return(
        p(
          class = "muted", "Save a prediction as A, edit or load another profile, then Predict scenario B."
        )
      )
    }
    a <- scenario_a()$probability
    tagList(
      p(class = "scenario-a", paste("Scenario A:", pct(a))),
      if (!is.null(current())) {
        b <- current()$probability
        tagList(
          p(class = "scenario-b", paste("Scenario B:", pct(b))),
          p(
            id = "scenario_difference", tags$strong(sprintf("B - A: %+.1f percentage points", 100 * (b - a)))
          )
        )
      }
```

```
    )
  })
  output$scenario_plot <- renderPlot({
    req(current())
    d <- data.frame(scenario = "B: current", probability = current()$probability)
    if (!is.null(scenario_a())) {
      d <- rbind(
        data.frame(scenario = "A: saved", probability = scenario_a()$probability),
        d
      )
    }
    ggplot2::ggplot(d, ggplot2::aes(scenario, probability, fill = scenario)) +
      ggplot2::geom_col(width = 0.5) +
      ggplot2::geom_text(
        ggplot2::aes(label = sprintf("%.1f%%", 100 * probability)),
        vjust = -0.4
      ) +
      ggplot2::scale_fill_manual(values = c(`A: saved` = "#D95F0E", `B: current` = "#2C7FB8")) +
      ggplot2::scale_y_continuous(
        limits = c(0, 1),
        labels = function(x) paste0(100 * x, "%")
      ) +
      ggplot2::theme_minimal(base_size = 12) +
      ggplot2::theme(legend.position = "none") +
      ggplot2::labs(x = NULL, y = "Predicted probability")
  })
  output$changed_inputs <- renderTable(
    {
      req(current(), scenario_a())
      changes <- scenario_changes(scenario_a()$profile, current()$profile, final_model)
      if (!nrow(changes)) {
        return(data.frame(Input = "No active inputs differ", A = "", B = ""))
      }
      changes
    },
    striped = TRUE
  )
  output$coefficients <- renderTable(rt$coefficients, digits = 6, striped = TRUE)
  output$dictionary <- renderTable(
    rt$dictionary[, c("variable", "role", "type", "units", "month")],
    striped = TRUE
  )
  performance_p <- reactive({
    req(input$perf_model)
    rt$predictions[[input$perf_model]]
  })
  performance_y <- rt$predictions$DEFAULT
  threshold <- reactive(threshold_metrics(performance_y, performance_p(), input$threshold))
  output$performance_summary <- renderUI({
    m <- rt$metrics[rt$metrics$family == input$perf_model, ]
    tagList(
      p(tags$strong(sprintf("AUC %.3f (95%% CI %.3f-%.3f)", m$AUC, m$Lower, m$Upper))),
      p(
        id = "fixed_metrics", sprintf("Log loss %.4f | Brier score %.4f", m$LogLoss, m$Brier)
      )
    )
  })
  output$threshold_summary <- renderUI({
    m <- threshold()
```

```
    tagList(
      div(
        class = "note", id = "decision_counts", tags$strong(
          paste(
            format(m["FN"], big.mark = ","),
            "missed defaults"
          )
        ),
        p(
          paste(
            format(m["FP"], big.mark = ","),
            "false alarms"
          )
        )
      ),
      p(
        paste(
          "Sensitivity", pct(m["Sensitivity"]),
          "| Specificity", pct(m["Specificity"])
        )
      ),
```

```
      p(
        id = "ppv", if (is.na(m["PPV"])) {
          "PPV: not estimable (no customers flagged)"
        } else {
          paste("PPV:", pct(m["PPV"]))
        }
      ),
      p(
        id = "flagged", paste(
          "Fraction flagged:", pct(m["Flagged"]),
          "| Accuracy:", pct(m["Accuracy"])
        )
      )
    )
  })
  output$confusion <- renderTable(
    {
      m <- threshold()
      data.frame(
        Recorded = c("Default", "No default"),
        Flagged = c(m["TP"], m["FP"]),
        `Not flagged` = c(m["FN"], m["TN"]),
        check.names = FALSE, row.names = NULL
      )
    },
    digits = 0
  )
  output$all_metrics <- renderTable(
    {
      x <- rt$metrics
      data.frame(
        Model = unname(model_labels[x$family]),
        AUC = x$AUC, `Log loss` = x$LogLoss, Brier = x$Brier, check.names = FALSE
      )
    },
    digits = 4,
    striped = TRUE
  )
  output$roc_plot <- renderPlot({
    r <- roc_object(performance_y, performance_p())
    d <- data.frame(FPR = 1 - r$specificities, Sensitivity = r$sensitivities)
    ggplot2::ggplot(d, ggplot2::aes(FPR, Sensitivity)) +
      ggplot2::geom_line(colour = "#2C7FB8", linewidth = 1) +
      ggplot2::geom_abline(linetype = "dashed", colour = "grey60") +
      ggplot2::geom_point(
        data = data.frame(FPR = 1 - threshold()["Specificity"], Sensitivity = threshold()["Sensitivity"]),
        colour = "#D95F0E", size = 4
      ) +
      ggplot2::labs(subtitle = sprintf("Orange point: threshold %.2f", input$threshold)) +
      ggplot2::coord_equal(
        xlim = c(0, 1),
        ylim = c(0, 1)
      ) +
      ggplot2::theme_minimal(base_size = 12) +
      ggplot2::labs(x = "1 - Specificity", y = "Sensitivity")
  })
  output$cal_plot <- renderPlot({
    d <- rt$calibration[[input$perf_model]]
    ggplot2::ggplot(d, ggplot2::aes(predicted, observed)) +
      ggplot2::geom_abline(linetype = "dashed", colour = "grey60") +
      ggplot2::geom_errorbar(
        ggplot2::aes(ymin = lower, ymax = upper),
        width = 0.015, colour = "#2C7FB8"
      ) +
      ggplot2::geom_point(colour = "#2C7FB8", size = 2) +
      ggplot2::coord_equal(
        xlim = c(0, 1),
        ylim = c(0, 1)
      ) +
      ggplot2::theme_minimal(base_size = 12) +
```

```
        ggplot2::labs(x = "Mean predicted probability", y = "Observed default proportion")
    })
  }
  shinyApp(ui, server)
}

# Source only defines functions. Explicit command-line actions are opt-in.
if (sys.nframe() == 0L) {
  arguments <- commandArgs(trailingOnly = TRUE)
  if (length(arguments)) {
```

```
    creditlens(arguments[1])
  }
}
```